# KÉTKEREKŰ ÖNKIEGYENSÚLYOZÓ ROBOT 3D DINAMIKAI MODELLEZÉSE ÉS IRÁNYÍTÁSA KLASSZIKUS ÉS TANULÁSALAPÚ SZABÁLYOZÁSI MÓDSZEREKKEL

**Várkonyi Borisz**

R83C3F
Mechatronikai mérnöki alapképzési szak
Nappali
Szent István Campus

**Dr. Erdélyi Viktor Ferenc**

Egyetemi Adjunktus
MATE, Mechatronika Tanszék
Szent István Campus

2026

# 1. fejezet

# Bevezetés

Ebben a fejezetben bemutatom a dolgozatban tárgyalt problémát és annak relevanciáját.

## 1.1. Téma jelentősége

Napjainkban a mesterséges intelligencia és a tanulásalapú módszerek rohamos fejlődése új lehetőségeket biztosít a robotok szabályozástechnikai alkalmazásaiban. A megerősítéses tanulás (reinforcement learning) lehetőséget biztosít arra, hogy a klasszikus szabályozási módszerek mellett alternatív, tanulásalapú szabályozókat alkalmazzunk, illetve a két megközelítést akár hibrid módon kombináljuk.

Az általam választott kutatási témában egy klasszikus, kaszkádolt szabályozási struktúrát hasonlítok össze egy Soft Actor-Critic (SAC) algoritmuson alapuló megerősítéses tanulási szabályozóval.

A kutatásom során MATLAB / Simulink környezetben létrehoztam egy kétkerekű fordított inga felépítésű robot (Inverted Pendulum On Wheels) nemlineáris matematikai modelljét, majd szintén ebben a környezetben készítettem hozzá egy kaszkádolt PID szabályzót, és egy megerősítéses tanulás alapút. A két szabályzó összehasonlításához létrehoztam továbbá egy általánosított teljesítményvizsgáló és kiértékelő algoritmust (benchmarkot) mely a releváns, megfigyelni kívánt paramétereket összesíti, és MATLAB környezetben diagrammokat generál.

## 1.2. Célkitűzés

A projekt célja egy összehasonlító analízis elkészítése a bevezetőben említett két algoritmus közt. Ennek lépései a következők voltak:

- A kétkerekű fordított inga robot nemlineáris matematikai modelljének megalkotása;
- Kaszkádolt PID vezérlő megtervezése és elkészítése.
- A PID szabályzó paramétereinek ideális értékeinek megállapítása kézi hangolással.
- A megfelelő környezet kialakítása a reinforcement learning szabályzóhoz.

- A reward (jutalom) függvény megtervezése és implementálása.
- Az ágens több iterációban való tanítása a jutalomfüggvény finomhangolásával.
- A betanított ágens tesztelése és validálása
- Összehasonlító benchmark tervezése és futtatása

# 2. fejezet

# Szakirodalom áttekintés

Dolgozatom ezen fejezete szakirodalom feldolgozás segítségével tekinti át a jelenleg is alkalmazott szabályzókat, PID szabályzó hangolási módszereket, tanítási módszereket, algoritmusokat és a jutalomfüggvény tervezési metódusokat. Az áttekintés kitér továbbá a jelen projekt számára releváns benchmark tervezési módszerekre is.

## 2.1. Kétkerekű önkiegyensúlyozó robotok

### 2.1.1. A fordított inga szabályozási problémája

Az alulaktuált mechanikai rendszereket (UMS) gyakran ideális tesztkörnyezetnek tekintik új szabályozási algoritmusok tervezéséhez és teszteléséhez, mivel nemlineáris és összetett dinamikai tulajdonságokkal rendelkeznek, valamint az általánosított koordinátáik száma meghaladja a rendelkezésre álló vezérlőbemenetek számát [1].

A kerekes robotok a lábas robotokhoz képest általában egyszerűbb mechanikai felépítésűek, és működésük is energiahatékonyabb lehet. A kétkerekű önkiegyensúlyozó robotoknál a két kerék egy közös tengelyen helyezkedik el, miközben a robot tömegközéppontja a tengely fölött található. Emiatt a rendszer statikusan instabil, ezért az egyensúly megtartásához folyamatos aktív szabályozás szükséges [1].

A kétkerekű kialakítás előnye, hogy a robot kis helyen is jól manőverezhető, és akár helyben is képes megfordulni. A kis alapterület miatt jól használható beltéri környezetben is. Az aktív egyensúlyozás miatt külső zavarásokra is képes reagálni, illetve lejtős felületen a test megdöntésével tudja megtartani az egyensúlyát [1].

A szabályozást az is nehezíti, hogy a rendszer nyílt hurokban instabil, alulaktuált, és a dőlésszög valamint az előremeneti sebesség dinamikája egymással össze van kapcsolva. Emiatt például előreinduláskor a robotnak először előre kell döntenie a testét, és csak ezután tud stabilan előre gyorsulni [1].

### 2.1.2. Kétkerekű önkiegyensúlyozó robotok dinamikai modellezése

A kétkerekű önkiegyensúlyozó robotok megfelelő szabályozásának egyik alapfeltétele a rendszer dinamikájának megfelelő matematikai leírása. A szakirodalomban több különböző modellezési módszer is elterjedt, ezek közül leggyakrabban a Newton-féle, a Lagrange-féle

és a Kane-módszert alkalmazzák [1, 2].

A Newton-féle megközelítés során a robot egyes merev testjeire ható erőket és nyomatékokat külön-külön vizsgálják. Ez jól szemlélteti a rendszer fizikai működését, azonban egy többtest-rendszer esetén a kölcsönható erők miatt a levezetés gyorsan összetetté válhat. A Lagrange-módszer ezzel szemben a rendszer kinetikus és potenciális energiájából indul ki, ezért több szabadságfokú mechanikai rendszerek modellezésénél gyakran alkalmazott módszer [2].

A kétkerekű önkiegyensúlyozó robot teljes mozgása három fő komponensre bontható: előremeneti mozgásra, a robot testének dőlésére, valamint a függőleges tengely körüli elfordulásra. A szakirodalomban ezek rendre a hosszirányú mozgás, a pitch és a yaw szabadságfokoknak felelnek meg. Bár a robot geometriai leírásához ennél több koordináta is szükséges lehet, a gördülési és geometriai kényszerek figyelembevételével a rendszer három független szabadságfokra redukálható [2].

A modellezés során fontos feltétel a kerekek és a talaj közötti csúszásmentes gördülés feltétele is. Ez kapcsolatot teremt a két kerék forgási sebessége, a robot előremeneti sebessége és a yaw szögsebessége között. A részletesebb modellek ezért nemcsak a dőlési mozgást vizsgálják, hanem az előremeneti és fordulási dinamika közötti kapcsolatot is figyelembe veszik [2].

A modell részletessége mindig függ a vizsgálat céljától. Egyszerűbb szabályozási feladatokhoz elegendő lehet egy kétdimenziós modell, ahol csak az előremeneti mozgást és a dőlésszöget vizsgálják. Amennyiben azonban a robot teljes mozgását, így a fordulást is vizsgálni szeretnénk, szükséges a yaw dinamika bevonása is. A jelen dolgozatban ezért egy három szabadságfokú, nemlineáris modell került kialakításra.

### 2.1.3. A szakirodalomban alkalmazott szabályozási módszerek

A kétkerekű önkiegyensúlyozó robotok szabályozására több különböző módszer is megtalálható a szakirodalomban. A leggyakrabban alkalmazott klasszikus megoldások közé tartozik a PID szabályozás és az állapot-visszacsatoláson alapuló LQR szabályozás. Ezek mellett több nemlineáris és intelligens szabályozási módszert is alkalmaznak, például sliding mode szabályozást, fuzzy logikát, neurális hálózatokat és megerősítéses tanulást [1, 3].

A PID szabályozás egyik előnye az egyszerű felépítés és az alacsony számítási igény. Kétkerekű önkiegyensúlyozó robotok esetén gyakran több egymásba ágyazott szabályozási kört alkalmaznak, ahol külön szabályozó felel például a dőlésszög, a sebesség vagy a pozíció követéséért [3]. Az LQR szabályozás ezzel szemben a rendszer állapottér-modelljére épül, és az állapotváltozók valamint a beavatkozás súlyozása alapján határozza meg a szabályozó paramétereit.

An és Li egy kétkerekű önkiegyensúlyozó roboton PID és LQR szabályozást is vizsgáltak MATLAB környezetben. A vizsgálat során külön kezelték az egyensúlyozást és a robot fordulását, majd összehasonlították a két szabályozási módszer eredményeit [4]. Ez jól mutatja, hogy ugyanazon robot szabályozási feladatára több különböző klasszikus megoldás is alkalmazható.

A nemlineáris szabályozási módszerek közül a sliding mode szabályozást is több kutatásban alkalmazták. Yuan és munkatársai egy kétkerekű önkiegyensúlyozó robot nemlineáris modelljéhez terveztek sliding mode szabályozót, majd annak működését szimulációval vizsgálták [5].

Az utóbbi években a klasszikus szabályozási módszerek mellett egyre nagyobb szerepet kapnak a tanulásalapú megoldások is. Ide tartoznak a neurális hálózaton és megerősítéses tanuláson alapuló szabályozók, amelyek egyik előnye, hogy összetett és nemlineáris rendszereken is alkalmazhatók. Ezek számítási igénye azonban általában nagyobb, mint a klasszikus PID vagy LQR szabályozóké [3].

## 2.2. Klasszikus szabályozási módszerek

### 2.2.1. PID és PD szabályozás

A PID szabályozó az egyik leggyakrabban alkalmazott klasszikus szabályozási módszer. Népszerűségének egyik oka az egyszerű felépítése, valamint az, hogy viszonylag kis számítási igény mellett sok különböző szabályozási feladatra alkalmazható [6, 3].

A PID szabályozó három tagból épül fel: arányos, integráló és deriváló tagból. A szabályozó általános alakja:

$$u(t) = K_p e(t) + K_i \int e(t)\,dt + K_d \frac{de(t)}{dt}, \tag{2.1}$$

ahol $e(t)$ a referenciaérték és a mért érték közötti hiba, $K_p$, $K_i$ és $K_d$ pedig a szabályozó hangolható paraméterei [6].

Az arányos tag közvetlenül a pillanatnyi hibára reagál. Nagyobb hiba esetén nagyobb beavatkozást eredményez. Az integráló tag a hiba időbeli összegét veszi figyelembe, ezért segítségével a tartós szabályozási hiba csökkenthető. A deriváló tag a hiba változási sebességére reagál, ezért elsősorban a rendszer csillapításában és a túllendülés csökkentésében van szerepe [6].

Nem minden szabályozási feladatnál szükséges mindhárom tag használata. Amennyiben az integráló tagra nincs szükség, PD szabályozás alkalmazható:

$$u(t) = K_p e(t) + K_d \frac{de(t)}{dt}. \tag{2.2}$$

Kétkerekű önkiegyensúlyozó robotoknál a PID és PD szabályozás is gyakran alkalmazott megoldás. A dőlésszög gyors stabilizálására például használható PD szabályozó, míg olyan szabályozási körökben, ahol tartós hiba is kialakulhat, az integráló tag alkalmazása is indokolt lehet [3].

### 2.2.2. Kaszkádolt szabályozási struktúrák

Összetettebb rendszereknél egyetlen szabályozó helyett több egymásba kapcsolt szabályozási kör is használható. Ezt kaszkádolt szabályozási struktúrának nevezzük. Ebben az esetben az egyik szabályozó kimenete egy következő szabályozási kör referenciaértékét adja.

A kétkerekű önkiegyensúlyozó robotoknál ez különösen jól alkalmazható, mivel egyszerre kell megoldani az egyensúly megtartását és a robot mozgásának szabályozását. A legbelső szabályozási kör általában a leggyorsabb dinamikájú változót, vagyis a robot dőlésszögét szabályozza. Ezen kívül helyezhető el például egy sebesség- vagy pozíciószabályozó kör [7, 3].

Nguyen egy kétkerekű önkiegyensúlyozó robot szabályozásánál szintén kaszkádolt PID struktúrát alkalmazott. A szabályozásban külön kezelte a robot egyensúlyozását és mozgását, majd a kialakított rendszert MATLAB/Simulink környezetben és valós roboton is vizsgálta [7].

A kaszkádolt kialakítás egyik előnye, hogy az egyes szabályozási feladatok külön kezelhetők. Így például a gyors dőlésszög-szabályozó először stabilizálja a robotot, miközben a lassabb külső szabályozási körök a kívánt sebesség vagy pozíció eléréséért felelnek.

### 2.2.3. PID szabályozók hangolási módszerei

A PID szabályozó megfelelő működéséhez szükséges a $K_p$, $K_i$ és $K_d$ paraméterek megfelelő beállítása. Bár maga a PID algoritmus egyszerű, a megfelelő paraméterek meghatározása összetettebb feladat lehet, mivel egyszerre több követelményt, például a gyors beállást, a kis túllendülést és a megfelelő stabilitást is figyelembe kell venni [6, 8].

A szakirodalomban több különböző PID hangolási módszer található. Az egyik legismertebb klasszikus módszer a Ziegler–Nichols hangolás, amely a rendszer válasza alapján ad kiindulási értékeket a szabályozó paramétereire. Emellett léteznek modellalapú, automatikus és különböző optimalizációs módszerek is [8].

A klasszikus hangolási módszerek előnye, hogy viszonylag egyszerűen alkalmazhatók és gyorsan adhatnak egy használható kiindulási pontot. A kapott paraméterek azonban nem minden esetben adják a kívánt működést, ezért a szabályozók további finomhangolása gyakran szükséges [6, 8].

A kézi hangolás során a szabályozó paramétereit több lépésben változtatják, és minden módosítás után megvizsgálják a rendszer válaszát. Szimulációs környezetben ez viszonylag gyorsan elvégezhető, ezért olyan rendszereknél is használható, ahol a szabályozási körök egymással kapcsolatban állnak és egyetlen általános hangolási módszer nehezen alkalmazható.

### 2.2.4. A klasszikus szabályozás előnyei és korlátai

A PID és más klasszikus szabályozók egyik legnagyobb előnye az egyszerű felépítés és az alacsony számítási igény. Emiatt valós idejű szabályozási feladatokban, akár kisebb teljesítményű mikrokontrollereken is jól alkalmazhatók. A PID szabályozó működése könnyen értelmezhető, a szabályozó paramétereinek hatása pedig közvetlenül vizsgálható [6, 3].

Kétkerekű önkiegyensúlyozó robotok esetén a PID szabályozás megfelelő hangolás mellett

jó eredményeket adhat, különösen egy meghatározott működési tartomány környezetében. A szakirodalomban számos valós roboton is alkalmaztak PID alapú szabályozást [3].

A klasszikus megoldások egyik hátránya viszont, hogy a szabályozó paraméterei a hangolás után általában rögzítettek. Ha a rendszer tulajdonságai jelentősen megváltoznak, például változik a robot tömege, a környezet vagy nagy külső zavarás lép fel, akkor a korábban meghatározott paraméterek már nem feltétlenül biztosítják ugyanazt a teljesítményt [3].

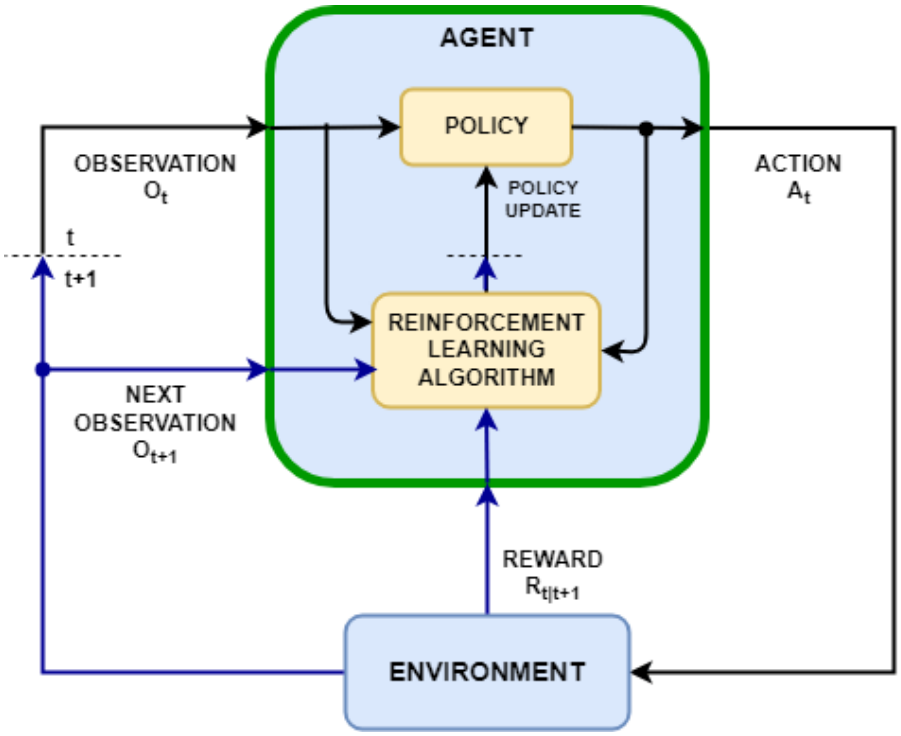
A kétkerekű önkiegyensúlyozó robot ezen kívül alapvetően nemlineáris rendszer, míg a hagyományos PID szabályozás nem használja ki közvetlenül a teljes nemlineáris modell információit. Ez nagyobb működési tartományban vagy összetettebb mozgások esetén korlátozhatja a szabályozó teljesítményét. Ennek ellenére az egyszerűsége, kis számítási igénye és könnyű megvalósíthatósága miatt továbbra is fontos viszonyítási alap más, összetettebb szabályozási módszerek vizsgálatánál [3].

## 2.3. Megerősítéses tanulás szabályozástechnikai alkalmazásokban

### 2.3.1. A megerősítéses tanulás alapjai

A megerősítéses tanulás (Reinforcement Learning, RL) egy olyan gépi tanulási módszer, ahol egy ágens a környezetével való folyamatos kapcsolat során tanulja meg a megfelelő viselkedést. Az ágens nem előre megadott szabályok alapján működik, hanem különböző beavatkozásokat próbál ki, majd az ezekre kapott visszajelzések alapján módosítja a működését [9, 10].

A tanulási folyamat során az ágens megfigyeli a környezet aktuális állapotát, majd ennek alapján kiválaszt egy akciót. Az akció hatására a környezet állapota megváltozik, az ágens pedig egy jutalomértéket kap. A jutalom azt mutatja meg, hogy az adott beavatkozás mennyire volt kedvező a kitűzött feladat szempontjából.




2.1. ábra. A megerősítéses tanulás alapvető ágens–környezet kapcsolata.

Az ágens célja nem feltétlenül az, hogy minden egyes időpillanatban a lehető legnagyobb jutalmat kapja, hanem hogy hosszabb idő alatt a lehető legnagyobb összesített jutalmat érje el. Ez azért fontos, mert egy rövid távon kedvezőtlen akció később akár jobb eredményhez is vezethet. A diszkontált összesített jutalom általánosan:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \tag{2.3}$$

ahol $\gamma$ a diszkonttényező, amely meghatározza, hogy a távolabbi jövőben kapott jutalmak milyen mértékben befolyásolják az ágens döntéseit [9].

Robotikai szabályozás esetén a környezet maga a szabályozott fizikai rendszer vagy annak szimulációs modellje, az ágens pedig a szabályozó szerepét tölti be. Az ágens a rendszer állapotából származó információk alapján állítja elő a beavatkozó jelet [10].

### 2.3.2. Állapot, akció és jutalom szerepe

A megerősítéses tanulási környezet egyik legfontosabb része annak meghatározása, hogy milyen információkat kapjon az ágens a rendszer aktuális állapotáról. Ezeket az információkat állapotnak vagy megfigyelésnek nevezzük. Robotikai alkalmazásokban ilyen információ lehet például egy pozíció, sebesség, gyorsulás, szöghelyzet vagy szögsebesség [10].

Az ágens a rendelkezésére álló megfigyelések alapján választja ki az akciót. Szabályozástechnikai feladatokban az akció általában közvetlen kapcsolatban áll a rendszer beavatkozó jelével. Ilyen lehet például egy motorra adott feszültség, erő vagy hajtónyomaték. Az ágens működését a policy, vagyis a tanult döntési stratégia határozza meg:

$$a_t \sim \pi(a_t|s_t), \tag{2.4}$$

ahol $s_t$ a rendszer állapota, $a_t$ pedig az ágens által választott akció az adott időpillanatban.

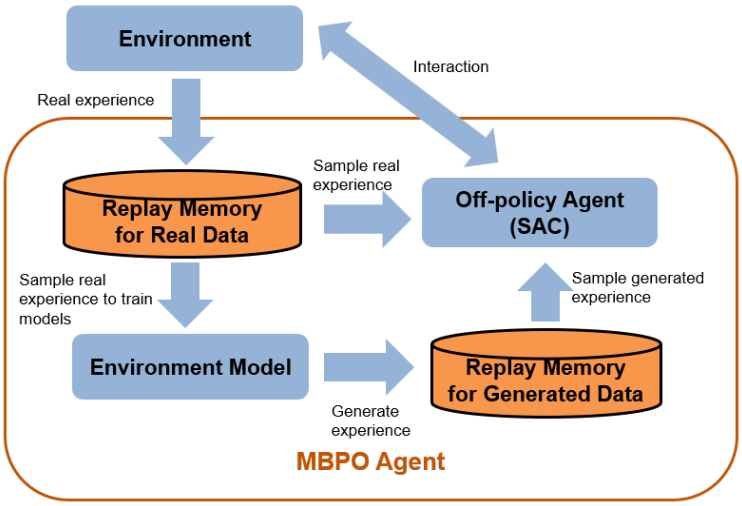
A harmadik alapvető elem a jutalomfüggvény. Ennek feladata, hogy számszerű visszajelzést adjon az ágens számára arról, hogy az adott állapot és beavatkozás mennyire felel meg a szabályozási célnak. Egy önkiegyensúlyozó robot esetén például pozitívan értékelhető a kis dőlésszög és a referenciaérték megfelelő követése, míg büntethető a nagy dőlés vagy a túl nagy beavatkozó nyomaték.

A jutalomfüggvény kialakítása ezért jelentősen befolyásolja, hogy az ágens milyen viselkedést tanul meg. Egy nem megfelelő jutalomfüggvény esetén az ágens olyan megoldást is találhat, amely matematikailag nagy jutalmat eredményez, de valós szabályozási szempontból nem kívánatos [10].

### 2.3.3. Megerősítéses tanulás folytonos szabályozási feladatokban

A szabályozástechnikai alkalmazások jelentős részében az állapotváltozók és a beavatkozó jelek nem csak néhány előre meghatározott értéket vehetnek fel, hanem folytonos tartományban változnak. Egy villanymotor hajtónyomatéka például nem csak néhány diszkrét érték közül választható, hanem egy adott minimum és maximum érték között folytonosan változtatható.

Ez különbséget jelent az egyszerűbb, diszkrét akcióterű megerősítéses tanulási feladatokhoz képest. Diszkrét esetben az ágens például néhány előre meghatározott akció közül választhat, míg folytonos akciótér esetén egy tetszőleges valós értéket kell meghatároznia az engedélyezett tartományon belül.




2.2. ábra. Példa folytonos beavatkozási terű megerősítéses tanulási feladatra egy fordított inga rendszerén.

A folytonos akcióterű problémák kezelésére több olyan megerősítéses tanulási algoritmust fejlesztettek ki, amelyek közvetlenül képesek folytonos beavatkozó jelek előállítására. Ilyen például a Deep Deterministic Policy Gradient (DDPG), amelyet több folytonos fizikai szabályozási feladaton, többek között fordított inga rendszeren is vizsgáltak [11].

Robotikai alkalmazásokban a folytonos akciótér különösen fontos, mivel a motorok által

előállított erő vagy nyomaték általában szintén folytonos mennyiség. A tanulási algoritmusnak ezért nemcsak azt kell meghatároznia, hogy milyen irányú beavatkozás szükséges, hanem annak megfelelő nagyságát is.

A folytonos állapot- és akcióterek kezelése ugyanakkor nehezebbé teszi a tanulási feladatot, mivel az ágensnek lényegesen nagyobb számú lehetséges állapot és beavatkozás között kell megfelelő működést megtanulnia [10, 11]. Emiatt ezeknél a feladatoknál gyakran neurális hálózatokat használnak a policy és más szükséges függvények közelítésére.

A dolgozatban vizsgált kétkerekű önkiegyensúlyozó robot szintén ilyen folytonos szabályozási problémát jelent, mivel a robot állapotváltozói folytonos értékeket vesznek fel, a szabályozó kimenete pedig közvetlenül a motorok közös hajtónyomatékához kapcsolódik. Emiatt olyan megerősítéses tanulási algoritmus alkalmazása szükséges, amely folytonos akciótér kezelésére is alkalmas.

## 2.4. Soft Actor-Critic algoritmus

### 2.4.1. Actor–Critic alapelv

Az Actor–Critic módszerekben két külön feladatot ellátó elem működik együtt. Az actor feladata annak meghatározása, hogy az ágens az aktuális állapotban milyen akciót hajtson végre, míg a critic az actor által választott akció minőségét értékeli [9].

Az actor tehát a policy-t, vagyis az ágens döntési stratégiáját valósítja meg:

$$a_t \sim \pi_\theta(a_t|s_t), \tag{2.5}$$

ahol $s_t$ az aktuális állapot, $a_t$ a kiválasztott akció, $\pi_\theta$ pedig az actor által meghatározott policy.

A critic ezzel párhuzamosan azt próbálja megbecsülni, hogy egy adott állapotban végrehajtott akció hosszabb távon mennyire kedvező. Ez például az állapot–akció érték-függvénnyel írható le:

$$Q(s, a). \tag{2.6}$$

Az actor a critic visszajelzése alapján módosítja a saját működését. A két hálózat ezért egymással párhuzamosan tanul: az actor egyre jobb akciókat próbál választani, a critic pedig egyre pontosabban próbálja meghatározni ezek várható értékét.

Az Actor–Critic módszerek egyik előnye, hogy folytonos akcióterű problémákra is jól alkalmazhatók. Ez szabályozástechnikai feladatoknál különösen fontos, mivel a beavatkozó jel, például egy motor nyomatéka vagy feszültsége általában folytonos értéket vesz fel.

### 2.4.2. A Soft Actor-Critic működése

A Soft Actor-Critic (SAC) egy modellfüggetlen, off-policy Actor–Critic megerősítéses tanulási algoritmus, amelyet elsősorban folytonos akcióterű problémák megoldására fejlesztettek ki [12].

Az off-policy működés azt jelenti, hogy az ágens a korábban összegyűjtött tapasztalatokat is újra fel tudja használni a tanítás során. A tapasztalatok egy úgynevezett replay bufferben kerülnek tárolásra, amelyből a tanítás során mintákat választ ki az algoritmus.

Emiatt ugyanaz a tapasztalat többször is felhasználható, ami javíthatja a tanulás minta-hatékonyságát [12].

A SAC egyik fontos tulajdonsága, hogy nem kizárólag a kapott jutalom maximalizálására törekszik. A célfüggvényben megjelenik a policy entrópiája is:

$$J(\pi) = \sum_t \mathbb{E}\left[r(s_t, a_t) + \alpha \mathcal{H}\left(\pi(\cdot|s_t)\right)\right], \tag{2.7}$$

ahol $r(s_t, a_t)$ az adott lépéshez tartozó jutalom, $\mathcal{H}$ a policy entrópiája, $\alpha$ pedig az entrópia és a jutalom közötti súlyozást meghatározó paraméter [12].

Az entrópia növelése azt eredményezi, hogy az ágens a tanítás során nem mindig ugyanazt az akciót választja, hanem több különböző lehetőséget is kipróbál. Ez segíti a környezet felfedezését, és csökkenti annak esélyét, hogy az ágens túl korán egy nem megfelelő megoldásnál maradjon.

A SAC stochasticus, vagyis valószínűségi policy-t használ. Az actor ezért nem közvetlenül egyetlen fix akciót határoz meg, hanem egy valószínűségi eloszlást, amelyből az akció kiválasztható. A tanítás előrehaladtával ez az eloszlás olyan akciók felé tolódik, amelyek magasabb hosszú távú jutalmat eredményeznek.

Az eredeti SAC algoritmus egyik célja az volt, hogy a korábbi deep reinforcement learning módszerekhez képest stabilabb tanulást és jobb minta-hatékonyságot biztosítson. Haarnoja és munkatársai folytonos szabályozási benchmarkokon vizsgálták az algoritmust, ahol a SAC több korábbi on-policy és off-policy módszerhez képest is jó eredményeket ért el [12].

A SAC későbbi változatában az entrópia súlyozására használt $\alpha$ paraméter automatikus hangolását is bevezették. Ez csökkenti annak szükségességét, hogy ezt a paramétert minden új problémához kézzel kelljen meghatározni [13]. A SAC-ot robotikai feladatokon is vizsgálták, többek között négylábú robot mozgásának és robotkéz manipulációs feladatainak tanítására. Ezek az eredmények azt mutatták, hogy az algoritmus összetett, folytonos szabályozási problémák esetén is alkalmazható [13].

A jelen dolgozat szempontjából a SAC egyik fontos tulajdonsága, hogy közvetlenül képes folytonos beavatkozó jel előállítására. Ez lehetővé teszi, hogy a tanult policy egy előre meghatározott tartományon belül közvetlenül adja meg a robot számára szükséges hajtónyomatékot.

## 2.5. Klasszikus és tanulásalapú szabályozók összehasonlítása

### 2.5.1. Szabályozási teljesítményt jellemző mérőszámok

Különböző szabályozási módszerek összehasonlításánál fontos, hogy az értékelés azonos körülmények és előre meghatározott mérőszámok alapján történjen. Önmagában az, hogy egy rendszer stabilan működik, nem elegendő annak eldöntésére, hogy az egyik szabályozó jobb-e a másiknál. Figyelembe kell venni többek között a referenciajel követésének pontosságát, a beállási időt, a maximális eltérést és a szükséges beavatkozás nagyságát is [3].

A szabályozási hiba jellemzésére használható például a négyzetes középértékből számított RMSE:

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{k=1}^{N}(x_{\mathrm{ref},k} - x_k)^2}. \tag{2.8}$$

Az RMSE egyetlen értékkel jellemzi, hogy a vizsgált jel a teljes mérési idő alatt átlagosan mennyire tért el a referenciaértéktől. Önkiegyensúlyozó robot esetén külön vizsgálható például a dőlésszög, a sebesség és a pozíció hibája.

Fontos mérőszám továbbá a maximális eltérés. Dőlésszög esetén ez megmutatja, hogy a robot a vizsgálat során legfeljebb milyen messz került a függőleges helyzettől:

$$\theta_{\max} = \max|\theta(t)|. \tag{2.9}$$

A beállási idő azt mutatja meg, hogy egy kezdeti eltérés vagy referenciajel-változás után mennyi idő szükséges ahhoz, hogy a rendszer egy meghatározott hibahatáron belül stabilizálódjon. Vizsgálható továbbá a túllendülés és a végső, állandósult hiba is.

A szabályozó teljesítményének értékelésénél a beavatkozó jel nagysága sem hagyható figyelmen kívül. Két szabályozó hasonló követési pontosság mellett jelentősen eltérő motor-nyomatékot igényelhet. Emiatt vizsgálható a maximális hajtónyomaték, annak RMS értéke, illetve a teljes vizsgálat alatt felhasznált beavatkozási energia közelítő értéke.

Klasszikus és tanulásalapú szabályozó összehasonlításánál különösen fontos, hogy mindkét algoritmus ugyanazon dinamikai modellen, azonos kezdeti állapotokkal, referenciajelekkel és beavatkozási korlátokkal kerüljön tesztelésre. Így a kapott eltérések nagyobb valószínűséggel magából a szabályozási módszerből adódnak, nem pedig a vizsgálati körülmények különbségéből.

### 2.5.2. Robusztusság, számítási igény és valós idejű alkalmazhatóság

Egy szabályozó vizsgálatánál a névleges működés mellett annak robusztussága is fontos szempont. A valós rendszerben a kezdeti állapot, a modellparaméterek és a környezeti hatások sem tekinthetők mindig pontosan ismertnek. Emiatt érdemes a szabályozókat eltérő kezdeti dőlésszögek, külső zavarások és módosított modellparaméterek mellett is vizsgálni [14, 3].

A klasszikus PID szabályozók egyik előnye, hogy egyszerű felépítésük miatt kis számítási igénnyel rendelkeznek, így valós idejű alkalmazásuk általában könnyen megoldható. Hátrányuk lehet ugyanakkor, hogy a rögzített szabályozóparaméterek miatt a rendszer jelentős megváltozása esetén a korábban megfelelő hangolás már nem feltétlenül ad azonos teljesítményt [3].

A megerősítéses tanuláson alapuló szabályozók képesek összetett, nemlineáris szabályozási stratégia megtanulására, és megfelelő tanítás esetén változó körülmények között is jó teljesítményt nyújthatnak. A szakirodalomban kétkerekű önkiegyensúlyozó robot esetén is vizsgáltak RL-alapú és klasszikus PID szabályozókat külső zavarás mellett [15]. A tanulásalapú megoldásoknál ugyanakkor a stabilitás és a korábban nem látott állapotokra

történő általánosítás továbbra is fontos vizsgálati kérdés [14].

A két szabályozási módszer összehasonlításánál a számítási igényt is figyelembe kell venni. A PID szabályozó néhány egyszerű matematikai műveletből számítja a beavatkozó jelet, míg egy neurális hálózaton alapuló szabályozó esetén minden szabályozási lépésben végre kell hajtani a hálózat előrehaladó számítását. Emiatt a tanulásalapú szabályozó végrehajtási ideje és memóriaigénye általában nagyobb [3].

Fontos különbséget tenni a megerősítéses tanulási algoritmus betanításának és a már betanított szabályozó futtatásának számítási igénye között. A tanítás akár nagyszámú szimulációs lépést és jelentős számítási kapacitást igényelhet, azonban a valós robot működése során már csak a betanított policy kiértékelését kell elvégezni.

Valós idejű alkalmazhatóság szempontjából ezért az egyik fontos mérőszám a szabályozó egyetlen vezérlési lépésének végrehajtási ideje. A szabályozónak minden esetben a rendszer mintavételi idején belül kell előállítania a következő beavatkozó jelet. Ennek vizsgálatára mérhető az átlagos és maximális számítási idő, valamint a processzor-, memória- és szükség esetén a grafikus processzor terhelése.

A klasszikus és tanulásalapú szabályozók összehasonlításánál ezért nem csak az a kérdés, hogy melyik képes pontosabban szabályozni a rendszert. Egy valós roboton történő alkalmazás szempontjából ugyanilyen fontos a robusztusság, a szükséges beavatkozás nagysága, a számítási igény és az, hogy a szabályozó képes-e a szükséges időkorláton belül működni.

# 3. fejezet

# Anyag és módszer

A fejezetben bemutatom a saját munkát

## 3.1. A vizsgált rendszer mechanikai modellje

A fejezet célja bemutatni a rendszer mechanikai modelljét.

**Koordinátarendszer és előjel-konvenció**

A világ koordinátarendszere:

$$X : \text{előre},$$
$$Y : \text{balra},$$
$$Z : \text{felfelé}.$$

A robot helyzetét és orientációját az alábbi mennyiségekkel írjuk le:

- $x$: a keréktengely középpontjának $X$ koordinátája;
- $y$: a keréktengely középpontjának $Y$ koordinátája;
- $\psi$: a robot irányszöge a függőleges tengely körül;
- $\theta$: a főtest hosszirányú dőlésszöge;
- $\varphi_{\mathrm{L}}$: a bal kerék talajhoz viszonyított abszolút gördülési szöge;
- $\varphi_{\mathrm{R}}$: a jobb kerék talajhoz viszonyított abszolút gördülési szöge.

Előjel-konvenció:

- $\theta > 0$: a főtest teteje előrefelé dől;
- $\psi > 0$: felülről nézve a robot balra fordul, vagyis pozitív $Z$ tengely körüli elfordulás történik.

A teljes konfigurációs vektor:

$$q_{teljes} = \begin{bmatrix} x & y & \psi & \theta & \varphi_{\mathrm{L}} & \varphi_{\mathrm{R}} \end{bmatrix}^T. \tag{3.1}$$

**A modell fizikai paraméterei**

| Jel | Jelentés | Egység |
|---|---|---|
| $m_\mathrm{b}$ | a főtest tömege | kg |
| $m_\mathrm{w}$ | egy kerék tömege | kg |
| $r$ | keréksugár | m |
| $b$ | nyomtáv, vagyis a két kerék érintkezési síkja közötti távolság | m |
| $l$ | a keréktengely és a főtest tömegközéppontja közötti távolság | m |
| $I_x$ | a főtest tehetetlenségi nyomatéka a test $x$ tengelye körül | $\mathrm{kg\,m^2}$ |
| $I_y$ | a főtest tehetetlenségi nyomatéka a test $y$ tengelye körül | $\mathrm{kg\,m^2}$ |
| $I_z$ | a főtest tehetetlenségi nyomatéka a test $z$ tengelye körül | $\mathrm{kg\,m^2}$ |
| $J_\mathrm{a}$ | egy kerék tengelyirányú forgási tehetetlensége | $\mathrm{kg\,m^2}$ |
| $J_{z,\mathrm{w}}$ | egy kerék elfordulási mozgáshoz tartozó tehetetlensége | $\mathrm{kg\,m^2}$ |
| $g$ | gravitációs gyorsulás | $\mathrm{m\,s^{-2}}$ |
| $\tau_\mathrm{L}$ | bal kerék kerékoldali hajtónyomatéka | N m |
| $\tau_\mathrm{R}$ | jobb kerék kerékoldali hajtónyomatéka | N m |

**A redukált koordináta-rendszer**

$$q_r = \begin{bmatrix} s \\ \theta \\ \psi \end{bmatrix} \tag{3.2}$$

Így a sebességvektor:

$$\dot{q_r} = \begin{bmatrix} \dot{s} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} v \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} \tag{3.3}$$

- $v$: a robot előremeneteli sebessége;
- $\dot{\theta}$: a főtest dőlési szögsebessége;
- $\dot{\psi}$: a robot fordulási szögsebessége

**A keréktengely-középpont síkbeli kinematikája**

$$e_f = \begin{bmatrix} cos(\psi) \\ sin(\psi) \end{bmatrix}, \quad v_p = \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \tag{3.4}$$

Oldalirányú csúszás nélkül a sebességvektor a robot előreirányába mutat, ezért:

$$\mathrm{v}_P = ve_f \tag{3.5}$$

Ebből következik, hogy:

$$\dot{x} = v \cdot cos(\psi), \quad \dot{y} = v \cdot sin(\psi) \tag{3.6}$$

**A két kerék gördülési és fordulási kinematikája**

$$v_L = v - \frac{b}{2}\dot{\psi}, \quad v_R = v - \frac{b}{2}\dot{\psi} \tag{3.7}$$

Csúszásmentes gördülés esetén $v_i = r\omega_i = r\dot{\phi}_i$, ezért:

$$r\dot{\phi}_L = v - \frac{b}{2}\dot{\psi}, \qquad r\dot{\phi}_R = v + \frac{b}{2}\dot{\psi} \tag{3.8}$$

A két egyenlet összeadásából és kivonásából:

$$\boxed{v = \frac{r}{2}\left(\dot{\phi}_R + \dot{\phi}_L\right)}, \qquad \boxed{\dot{\psi} = \frac{r}{b}\left(\dot{\phi}_R - \dot{\phi}_L\right)}. \tag{3.9}$$

**Az abszolút és relatív kerékszög kapcsolata**

A relatív encoder-szög a kerék és a robot főtestének szöghelyzete közötti különbség:

$$\alpha_i = \phi_i - \theta. \tag{3.10}$$

Idő szerint deriválva:

$$\dot{\alpha}_i = \dot{\phi}_i - \dot{\theta}. \tag{3.11}$$

Mivel $\omega_i = \dot{\phi}_i$, ezért:

$$\boxed{\dot{\alpha}_L = \omega_L - \dot{\theta}, \qquad \dot{\alpha}_R = \omega_R - \dot{\theta}.} \tag{3.12}$$

**A főtest középpontjának helye**

A keréktengely középpontja:

$$\mathbf{p}_P = \begin{bmatrix} x \\ y \\ r \end{bmatrix} \tag{3.13}$$

A tengely és a tömegközéppont közötti $l$ hosszúságú vektor vízszintes vetülete $l\sin\theta$, függőleges vetülete $l\cos\theta$, ezért:

$$\mathbf{r}_{PG} = l\begin{bmatrix}\sin\theta\cos\psi\\ \sin\theta\sin\psi\\ \cos\theta\end{bmatrix} \tag{3.14}$$

Így:

$$\boxed{\mathbf{p}_G = \mathbf{p}_P + \mathbf{r}_{PG} = \begin{bmatrix}x + l\sin\theta\cos\psi\\ y + l\sin\theta\sin\psi\\ r + l\cos\theta\end{bmatrix}} \tag{3.15}$$

**A főtest tömegközéppontjának sebessége**

Az előző helyvektor idő szerinti deriváltja, felhasználva $\dot{x} = v\cos\psi$ és $\dot{y} = v\sin\psi$:

$$\dot{\mathbf{p}}_G = \begin{bmatrix}(v + l\cos\theta\,\dot{\theta})\cos\psi - l\sin\theta\,\dot{\psi}\sin\psi\\ (v + l\cos\theta\,\dot{\theta})\sin\psi + l\sin\theta\,\dot{\psi}\cos\psi\\ -l\sin\theta\,\dot{\theta}\end{bmatrix}. \tag{3.16}$$

A sebesség nagyságának négyzete:

$$\|\mathbf{v}_G\|^2 = \dot{x}_G^2 + \dot{y}_G^2 + \dot{z}_G^2. \tag{3.17}$$

Egyszerűsítés után:

$$\boxed{\|\mathbf{v}_G\|^2 = v^2 + 2lv\cos\theta\,\dot{\theta} + l^2\dot{\theta}^2 + l^2\sin^2\theta\,\dot{\psi}^2.} \tag{3.18}$$

**A hajtónyomatékok általánosított erői**

A két kerék hajtóereje $F_i = \tau_i/r$, ezért az előremeneti általánosított erő:

$$Q_v = \frac{\tau_L + \tau_R}{r} = \frac{2\tau_c}{r}. \tag{3.19}$$

A főtestre a kerékhajtással ellentétes reakciónyomaték hat, míg a két kerékerő különbsége $b/2$ erőkarral elfordító nyomatékot hoz létre:

$$Q_\theta = -(\tau_L + \tau_R) = -2\tau_c, \qquad Q_\psi = \frac{b}{2r}(\tau_R - \tau_L) = \frac{b}{r}\tau_d. \tag{3.20}$$

Így:

$$\boxed{\mathbf{Q} = \begin{bmatrix}Q_v\\ Q_\theta\\ Q_\psi\end{bmatrix} = \begin{bmatrix}2/r & 0\\ -2 & 0\\ 0 & b/r\end{bmatrix}\begin{bmatrix}\tau_c\\ \tau_d\end{bmatrix}} \tag{3.21}$$

### A rendszer teljes mozgási energiája

A főtest haladó és forgási energiája:

$$T_{b,t} = \frac{1}{2} m_b \left( v^2 + 2lv \cos\theta\, \dot{\theta} + l^2 \dot{\theta}^2 + l^2 \sin^2\theta\, \dot{\psi}^2 \right), \tag{3.22}$$

$$T_{b,r} = \frac{1}{2} I_y \dot{\theta}^2 + \frac{1}{2} \left( I_x \sin^2\theta + I_z \cos^2\theta \right) \dot{\psi}^2. \tag{3.23}$$

A két kerék haladó, saját tengely körüli és elfordulási energiája:

$$T_{w,t} = m_w v^2 + \frac{1}{4} m_w b^2 \dot{\psi}^2, \tag{3.24}$$

$$T_{w,a} = \frac{J_a}{r^2} v^2 + \frac{J_a b^2}{4r^2} \dot{\psi}^2, \qquad T_{w,z} = J_{z,w} \dot{\psi}^2. \tag{3.25}$$

Így a teljes mozgási energia:

$$\boxed{T = T_{b,t} + T_{b,r} + T_{w,t} + T_{w,a} + T_{w,z}.} \tag{3.26}$$

### Összevont tehetetlenségi együtthatók

A teljes kinetikus energiában a $v^2$, $\dot{\theta}^2$ és $\dot{\psi}^2$ tagok együtthatóit külön-külön összegyűjtjük:

$$\boxed{A = m_b + 2m_w + \frac{2J_a}{r^2}, \qquad C = I_y + m_b l^2} \tag{3.27}$$

$$\boxed{D(\theta) = I_x \sin^2\theta + I_z \cos^2\theta + m_b l^2 \sin^2\theta + \frac{m_w b^2}{2} + \frac{J_a b^2}{2r^2} + 2J_{z,w}.} \tag{3.28}$$

Mivel $D$ függ $\theta$-tól:

$$\boxed{D'(\theta) = 2 \sin\theta \cos\theta \left( I_x - I_z + m_b l^2 \right).} \tag{3.29}$$

### A teljes kinetikus energia redukált alakja

Az összes korábban meghatározott kinetikusenergia-tagot összeadva és az $A$, $C$, $D(\theta)$ mennyiségeket felhasználva:

$$T = T_{b,t} + T_{b,r} + T_{w,t} + T_{w,a} + T_{w,z}. \tag{3.30}$$

Az azonos sebességtagok összevonása után:

$$\boxed{T = \frac{1}{2} A v^2 + m_b l \cos\theta\, v\dot{\theta} + \frac{1}{2} C \dot{\theta}^2 + \frac{1}{2} D(\theta) \dot{\psi}^2.} \tag{3.31}$$

A vegyes $v\dot{\theta}$ tag az előremeneti és dőlési mozgás közötti csatolást írja le.

### Potenciális energia

A főtest tömegközéppontjának keréktengelyhez viszonyított magassága:

$$h = l \cos\theta. \tag{3.32}$$

A gravitációs potenciális energia:

$$V = m_b g h. \tag{3.33}$$

Ezért:

$$\boxed{V = m_b g l \cos\theta.} \tag{3.34}$$

**A Lagrange-függvény**

A Lagrange-függvény definíció szerint a kinetikus és potenciális energia különbsége:

$$L = T - V. \tag{3.35}$$

A korábban kapott energiákat behelyettesítve:

$$\boxed{L = \frac{1}{2}Av^2 + m_b l\cos\theta\, v\dot{\theta} + \frac{1}{2}C\dot{\theta}^2 + \frac{1}{2}D(\theta)\dot{\psi}^2 - m_b g l\cos\theta.} \tag{3.36}$$

A redukált rendszer független sebességváltozói:

$$\boxed{v, \qquad \dot{\theta}, \qquad \dot{\psi}.} \tag{3.37}$$

**A nemlineáris mozgásegyenletek**

A Lagrange–d'Alembert-egyenleteket a három redukált mozgásirányra alkalmazzuk, a korábban meghatározott $Q_v$, $Q_\theta$, $Q_\psi$ általánosított erőkkel.

$$\boxed{A\dot{v} + m_b l\cos\theta\,\ddot{\theta} - m_b l\sin\theta\,\dot{\theta}^2 = \frac{2\tau_c}{r}} \tag{3.38}$$

$$\boxed{m_b l\cos\theta\,\dot{v} + C\ddot{\theta} - \frac{1}{2}D'(\theta)\dot{\psi}^2 - m_b g l\sin\theta = -2\tau_c} \tag{3.39}$$

$$\boxed{D(\theta)\ddot{\psi} + D'(\theta)\dot{\theta}\dot{\psi} = \frac{b}{r}\tau_d.} \tag{3.40}$$

**Mátrixos alak**

A három mozgásegyenlet gyorsulási, nemlineáris és bemeneti tagjait külön csoportosítva:

$$\boxed{M(q_r)\ddot{q}_r + h(q_r, \dot{q}_r) = Bu.} \tag{3.41}$$

ahol

$$M = \begin{bmatrix} A & m_b l\cos\theta & 0 \\ m_b l\cos\theta & C & 0 \\ 0 & 0 & D(\theta) \end{bmatrix}, \qquad B = \begin{bmatrix} 2/r & 0 \\ -2 & 0 \\ 0 & b/r \end{bmatrix}. \tag{3.42}$$

Így a gyorsulások numerikusan:

$$\boxed{\ddot{q}_r = M\backslash(Bu - h)} \tag{3.43}$$

## 3.2. Az analitikus modell numerikus implementációja és validációja

A levezetett nemlineáris mozgásegyenletek numerikus vizsgálatához a modellt MATLAB/Simulink környezetben implementáltam. A modell célja egyrészt a matematikai

levezetés helyességének ellenőrzése, másrészt egy olyan gyorsan szimulálható növénymodell létrehozása volt, amely később a klasszikus és a megerősítéses tanuláson alapuló szabályozók fejlesztésének közös alapját képezi.

A redukált rendszer mátrixos alakja:

$$\mathbf{M}(\mathbf{q}_r)\ddot{\mathbf{q}}_r + \mathbf{h}(\mathbf{q}_r, \dot{\mathbf{q}}_r) = \mathbf{B}\mathbf{u} + \mathbf{Q}_{\text{vesz}}, \tag{3.44}$$

ahol

$$\mathbf{q}_r = \begin{bmatrix} s & \theta & \psi \end{bmatrix}^T, \qquad \dot{\mathbf{q}}_r = \begin{bmatrix} v & \dot{\theta} & \dot{\psi} \end{bmatrix}^T, \tag{3.45}$$

és a vezérlőbemenet:

$$\mathbf{u} = \begin{bmatrix} \tau_c & \tau_d \end{bmatrix}^T. \tag{3.46}$$

A numerikus számítás során az explicit mátrixinverz képzése helyett lineáris egyenletrendszer megoldása történik:

$$\ddot{\mathbf{q}}_r = \mathbf{M} \backslash \left( \mathbf{B}\mathbf{u} + \mathbf{Q}_{\text{vesz}} - \mathbf{h} \right). \tag{3.47}$$

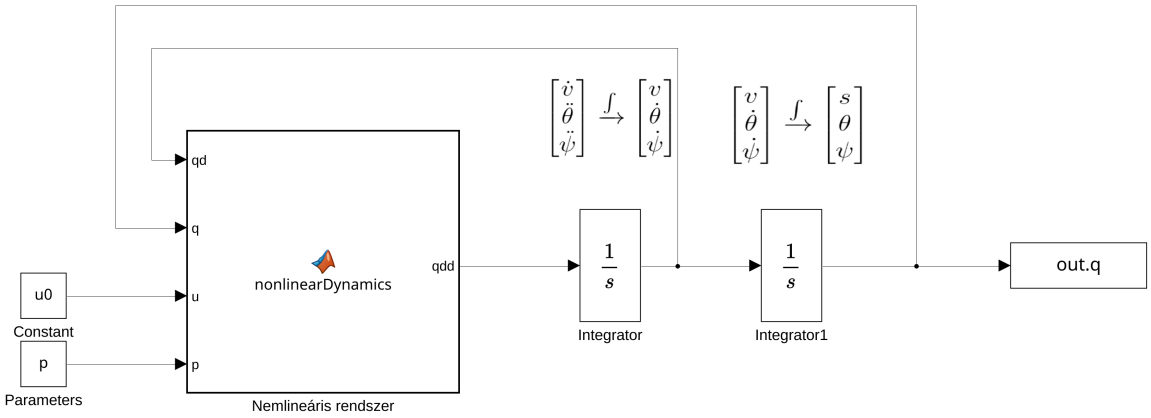
Ez numerikusan kedvezőbb, mint a $\mathbf{M}^{-1}$ explicit meghatározása.

### 3.2.1. Simulink implementáció

A Simulink modellben egy MATLAB Function blokk számítja a pillanatnyi gyorsulásvektort a rendszer aktuális állapota, a motorok nyomatékbemenetei és a fizikai paraméterek alapján. A számított gyorsulásokat két egymás után kapcsolt integrátor alakítja sebesség-, majd helyzetállapotokká:

$$\ddot{\mathbf{q}}_r \xrightarrow{\int} \dot{\mathbf{q}}_r \xrightarrow{\int} \mathbf{q}_r. \tag{3.48}$$

Az integrátorok kimenetei visszacsatolásra kerülnek a nemlineáris dinamikai modellhez, így minden időlépésben az aktuális állapot alapján történik a gyorsulások újraszámítása.




3.1. ábra. A nemlineáris analitikus robotmodell Simulink implementációja.

### 3.2.2. Disszipatív veszteségek modellezése

Az ideális mechanikai modell csúszásmentes gördülést feltételez, azonban önmagában nem tartalmaz energiaveszteséget. Emiatt vezérlés nélküli szimulációban a rendszer mechanikai energiája megmarad, és a főtest a függőleges instabil helyzetből eldőlve korlátlan ideig tovább lenghet, illetve többször átfordulhat.

A valós rendszer viselkedésének jobb közelítéséhez a modellben több disszipatív hatást is figyelembe vettem:

- motor- és csapágysúrlódás,
- kerék–talaj gördülési ellenállás,
- kis mértékű sebességfüggő légellenállás.

A motor és a főtest közötti relatív kerék-szögsebességek:

$$\dot{\alpha}_L = \frac{v - \frac{b}{2}\dot{\psi}}{r} - \dot{\theta}, \qquad \dot{\alpha}_R = \frac{v + \frac{b}{2}\dot{\psi}}{r} - \dot{\theta}. \tag{3.49}$$

A motor- és csapágysúrlódást viszkózus és simított Coulomb-tag összegeként modelleztem:

$$\tau_{f,i} = c_m \dot{\alpha}_i + \tau_{C,m} \tanh\left(\frac{\dot{\alpha}_i}{\omega_\varepsilon}\right), \qquad i \in \{L, R\}. \tag{3.50}$$

A hiperbolikus tangens alkalmazása a $\mathrm{sign}(\cdot)$ függvény numerikusan simított közelítését adja, így nulla sebesség környezetében elkerülhető a diszkontinuitás.

A két kerék hosszirányú sebessége:

$$v_L = v - \frac{b}{2}\dot{\psi}, \qquad v_R = v + \frac{b}{2}\dot{\psi}. \tag{3.51}$$

A gördülési ellenállás kerekenként:

$$F_{rr,i} = c_{rr} v_i + F_{C,rr} \tanh\left(\frac{v_i}{v_\varepsilon}\right). \tag{3.52}$$

A gördülési ellenállás által létrehozott általánosított erővektor:

$$\mathbf{Q}_{rr} = \begin{bmatrix} -(F_{rr,L} + F_{rr,R}) \\ 0 \\ -\frac{b}{2}(F_{rr,R} - F_{rr,L}) \end{bmatrix}. \tag{3.53}$$

A légellenállást egyszerű másodfokú közelítéssel vettem figyelembe:

$$F_{\mathrm{air}} = c_{\mathrm{air}} v |v|. \tag{3.54}$$

A teljes veszteségi általánosított erő:

$$\mathbf{Q}_{\mathrm{vesz}} = \mathbf{Q}_{\mathrm{motor}} + \mathbf{Q}_{rr} + \mathbf{Q}_{\mathrm{air}}. \tag{3.55}$$

A használt veszteségi paraméterek jelenleg becsült modellparaméterek; pontos azonosításuk a fizikai prototípus mérési eredményei alapján későbbi feladat.

### 3.2.3. Az analitikus modell validációja

A numerikus implementáció alapvető fizikai helyességének ellenőrzésére több nyílt hurkú vizsgálatot végeztem. A vizsgálatok során nem alkalmaztam aktív szabályozást, így a kapott válasz kizárólag a mechanikai modell dinamikájából származott.

3.2. táblázat. Az analitikus modell nyílt hurkú ellenőrző vizsgálatai.

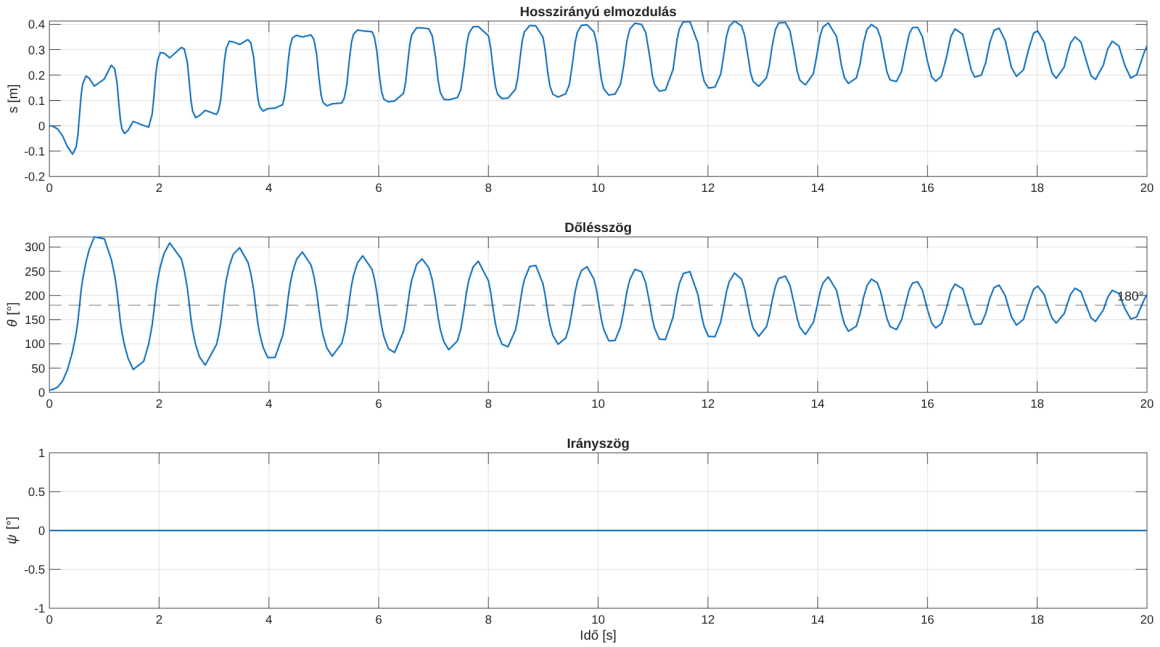
| **Vizsgálat** | **Kezdeti feltétel / bemenet** | **Tapasztalt viselkedés** |
|---|---|---|
| Instabil egyensúly | $\theta_0 = 2°$, $\tau_c = \tau_d = 0$ | A dőlésszög abszolút értéke növekedett, a függőleges helyzet instabilnak bizonyult. |
| Közös hajtónyomaték | $\tau_c > 0$, $\tau_d = 0$ | Hosszirányú mozgás és dőlés alakult ki, miközben $\psi = 0$ maradt. |
| Különbségi hajtónyomaték | $\tau_c = 0$, $\tau_d > 0$ | A robot irányszöge változott, miközben ideális kezdeti állapot esetén $s$ és $\theta$ változatlan maradt. |
| Csillapított szabadesés | $\theta_0 = 5°$, $\tau_c = \tau_d = 0$ | A főtest csillapodó lengések után a lefelé mutató, stabil egyensúlyi helyzethez közelített. |

#### Csillapított szabadesés

A veszteségi modell további ellenőrzéséhez a rendszert $\theta_0 = 5°$ kezdeti dőlésből, nulla motorbemenet mellett szimuláltam. A kapott válasz a 3.2. ábrán látható.

A főtest először eldől, majd a lefelé mutató $\theta = 180°$ stabil egyensúlyi helyzet körül leng. A motor-, csapágy- és gördülési veszteségek következtében a lengés amplitúdója fokozatosan csökken.

A hosszirányú elmozdulás szintén oszcilláló viselkedést mutat, ami a dőlési és hosszirányú dinamika közötti csatolás következménye. A különbségi hajtónyomaték hiánya miatt az irányszög a teljes vizsgálat alatt változatlan marad.




3.2. ábra. Az analitikus modell válasza $\theta_0 = 5°$ kezdeti dőlés és nulla motorbemenet esetén, a disszipatív veszteségek figyelembevételével. A rendszer a $\theta = 180°$ stabil egyensúlyi helyzethez tart.

## 3.3. Klasszikus kaszkád szabályozó kialakítása

A kétkerekű önkiegyensúlyozó robot szabályozásának első lépéseként egy klasszikus, kaszkádolt szabályozási struktúra került kialakításra.

A szabályozó célja az volt, hogy a robotot $\theta = 0$ dőlésszöghöz a legközelebb tartsa tehát stabilizálja a függőleges egyensúlyi helyzetet, a sebesség kövesse a kijelölt referenciaértéket, és a pozíciót a referenciaértéknek megfelelően megközelítse.

A kaszkádolt szabályozásban a dőlésszögszabályzó volt a legbelső, amely azért volt fontos, mert ennek kellett a leggyorsabban működnie és hatást kifejtenie a rendszerre. Ez után eggyel kijjebb elhelyezkedő kör volt a sebességszabályzó, majd legkívülre került a pozíciószabályzás. A szabályzó kimenete $\tau_c$ volt, mely nem más, mint a két kerék hajtónyomatékának közös komponense.

$$s_{\text{ref}} \rightarrow v_{\text{ref}} \rightarrow \theta_{\text{ref}} \rightarrow \tau_c \tag{3.56}$$

### 3.3.1. Belső dőlésszög-szabályozó

Először csak az egyensúlyozási problémát oldottam meg. Ehhez egy arányos-deriváló szabályozót hoztam létre, amelyben az arányos tag a dőlésszög-hibára reagál, míg a deriváló tag csillapítja a mozgást, valamint csökkenti a túllendülést és az oszcillációt.

$$e_\theta = \theta - \theta_{\text{ref}} \tag{3.57}$$

A legbelső szabályozási kör szabályozási törvénye az alábbi:

$$\tau_c^* = K_{p,\theta} e_\theta + K_{d,\theta} \dot{\theta}. \tag{3.58}$$

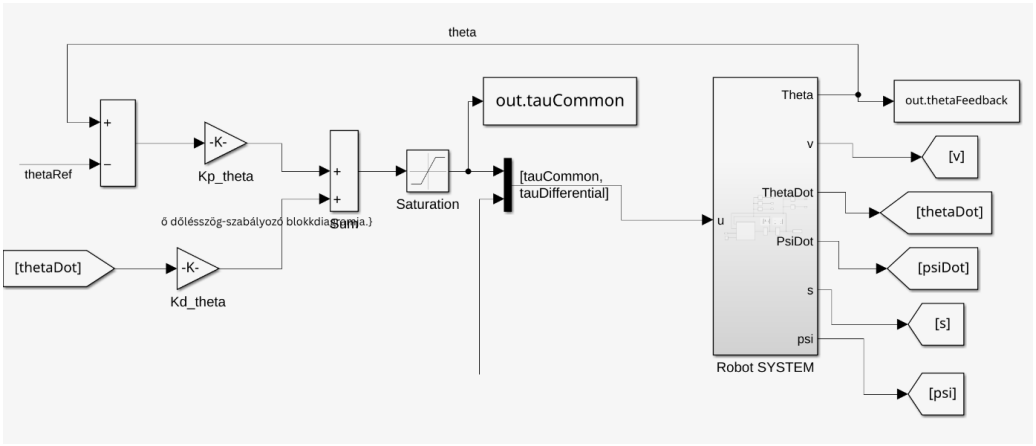
A $\tau_c^*$ jelölésben szereplő csillag azt jelzi, hogy ez még a szabályozó által előállított, szaturáció előtti beavatkozó jel. A szabályozás során alkalmazott paraméterek:

$$K_{p,\theta} = 2, \qquad K_{d,\theta} = 0{,}2. \tag{3.59}$$

A közös hajtónyomatékot a hajtás fizikai korlátainak figyelembevétele érdekében $\pm 2\,\mathrm{Nm}$ értékre korlátoztam:

$$|\tau_c| \leq 2\,\mathrm{Nm}. \tag{3.60}$$

A belső dőlésszög-szabályozó felépítése a 3.7. ábrán látható.




3.3. ábra. A belső dőlésszög-szabályozó blokkdiagramja.

### 3.3.2. Sebességszabályozó kör

Miután a belső dőlésszög-szabályozó megfelelően stabilizálta a robotot, a következő lépés az előremeneti sebesség szabályozása volt. A sebességszabályozó nem közvetlenül a motor nyomatékát határozza meg, hanem a belső szabályozási kör számára állít elő dőlésszög-referenciaértéket.

A sebességhiba az aktuális és a kívánt sebesség különbségéből adódik:

$$e_v = v_{\mathrm{ref}} - v. \tag{3.61}$$

A sebességszabályozó arányos és integráló tagot tartalmazott. Az arányos tag a pillanatnyi sebességhibára reagál, míg az integráló tag célja az volt, hogy a tartós sebességeltérést csökkentse.

A szabályozó kimenete a dőlésszög referenciaértéke:

$$\theta_{\mathrm{ref}}^* = K_{p,v} e_v + K_{i,v} x_I. \tag{3.62}$$

A sebességszabályozó paramétereit a szimulációk során fokozatosan hangoltam. A végül alkalmazott értékek:

$$K_{p,v} = 0{,}25, \qquad K_{i,v} = 0{,}3. \tag{3.63}$$

A kezdeti vizsgálatok során tiszta integráló tag alkalmazása mellett kis sebességek környezetében tartós lengés jelentkezett. Ez főként a modellben szereplő súrlódási hatások miatt alakult ki. Ennek csökkentésére a tiszta integrátor helyett egy szivárgó integráló tagot alkalmaztam:

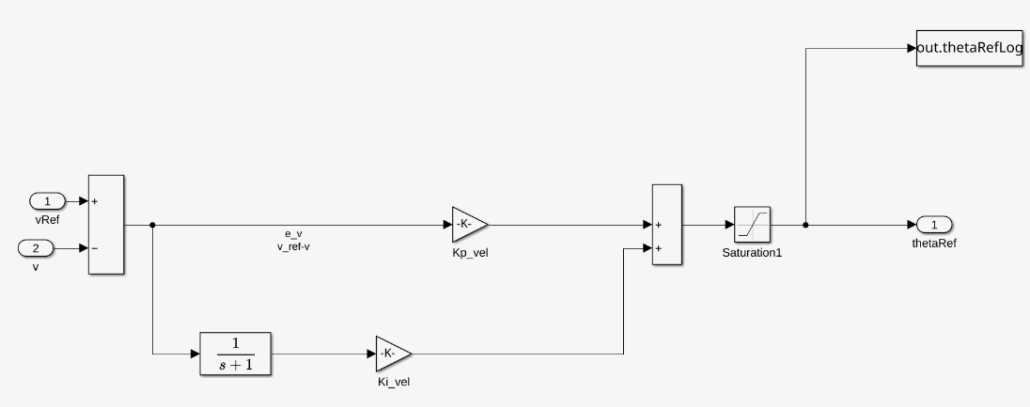
$$X_I(s) = \frac{1}{s+1} E_v(s). \tag{3.64}$$

Ez a megoldás lehetővé tette, hogy az integráló hatás megmaradjon, ugyanakkor csökkentette a nulla sebesség környezetében kialakuló oszcillációt.

A sebességszabályozó által előállított dőlésszög-referencia szintén korlátozásra került. A túl nagy referenciaérték túl nagy dőlést és ezzel instabil viselkedést eredményezhetne, ezért a maximális referenciaértéket

$$|\theta_{\text{ref}}| \leq 8^\circ \tag{3.65}$$

értékben határoztam meg.




3.4. ábra. A középső sebesség-szabályozó blokkdiagramja.

### 3.3.3. Pozíciószabályozó kör

A kaszkádolt szabályozási struktúra legkülső körét a pozíciószabályozó alkotja. Ennek feladata, hogy a robot aktuális pozíciója és az előírt referenciaérték közötti eltérés alapján meghatározza a sebességszabályozó számára szükséges sebesség-referenciaértéket.
A pozícióhiba:

$$e_s = s_{\text{ref}} - s. \tag{3.66}$$

A pozíciószabályozó kialakításánál egy arányos szabályozást alkalmaztam. A szabályozó kimenete közvetlenül a következő, sebességszabályozó kör referenciaértéke:

$$v^*_{\text{ref}} = K_{p,s} e_s. \tag{3.67}$$

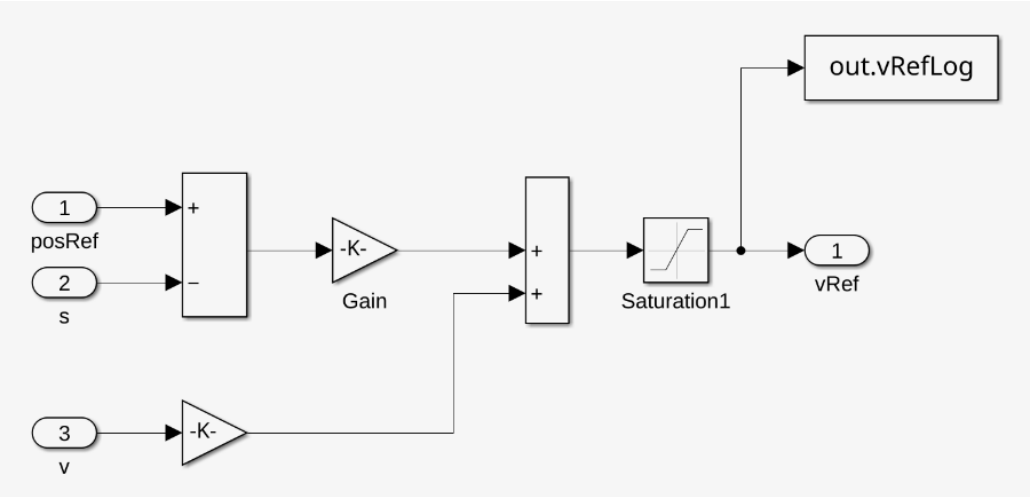
A szabályozó paraméterének hangolása szimulációs vizsgálatok alapján történt. A végül alkalmazott érték:

$$K_{p,s} = 0{,}05. \tag{3.68}$$

A pozícióhiba növekedésével a szabályozó egyre nagyobb sebesség-referenciaértéket állítana elő. A robot túlzott sebességének és az ebből következő nagy dőlésszög-referenciák elkerülése érdekében ezért a sebesség-referencia értékét korlátoztam:

$$|v_{\text{ref}}| \leq 0{,}2\,\text{m/s}. \tag{3.69}$$

A pozíciószabályozó ezért nem közvetlenül a motorok nyomatékára hat, hanem a sebességszabályozó számára hoz létre referenciaértéket. A sebességszabályozó ebből dőlésszög-referenciát állít elő, amelyet végül a belső dőlésszög-szabályozó alakít át közös hajtónyomatékká.




3.5. ábra. A külső pozíció-szabályozó blokkdiagramja.

### 3.3.4. Irányszög-sebesség szabályozása

A hosszirányú mozgás szabályozása mellett szükséges volt a robot fordulásának vezérlése is. Ezt a két kerék hajtónyomatékának különbségi komponensével, vagyis a $\tau_d$ nyomatékkal valósítottam meg.

A szabályozás során nem közvetlenül a $\psi$ irányszöget, hanem annak idő szerinti deriváltját, vagyis a fordulási szögsebességet szabályoztam. A hibajel:

$$e_{\dot{\psi}} = \dot{\psi}_{\text{ref}} - \dot{\psi}. \tag{3.70}$$

A különbségi nyomaték meghatározásához arányos szabályozót alkalmaztam:

$$\tau_d^* = K_{p,\psi} e_{\dot{\psi}}. \tag{3.71}$$

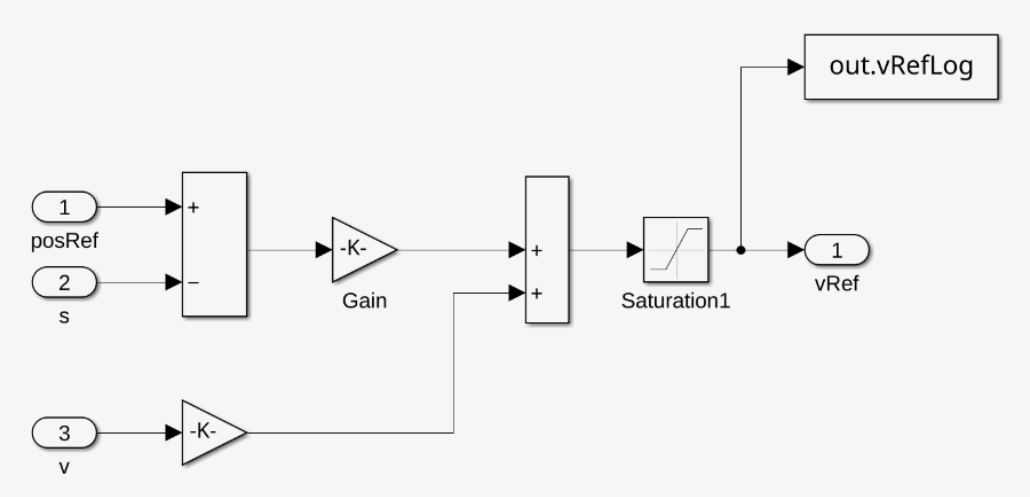
A szabályozó arányos erősítése:

$$K_{p,\psi} = 0{,}02. \tag{3.72}$$

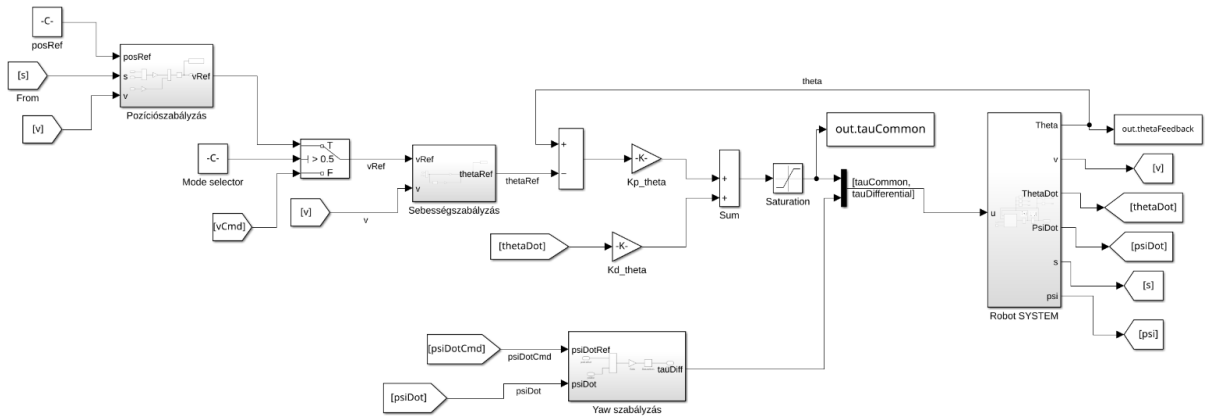
A túl nagy fordulási nyomaték elkerülése érdekében a különbségi nyomatékot szintén korlátoztam:

$$|\tau_d| \leq 0{,}2\,\mathrm{Nm}. \tag{3.73}$$

A $\tau_c$ közös hajtónyomaték így elsősorban az egyensúlyozást és a hosszirányú mozgást, míg a $\tau_d$ különbségi nyomaték a robot fordulását határozza meg.




3.6. ábra. Az Irányszög-Sebesség-szabályozó blokkdiagramja.




3.7. ábra. A teljes kaszkádolt szabályozó blokkdiagramja.

## 3.4. Megerősítéses tanulási környezet kialakítása

### 3.4.1. A szabályozási feladat meghatározása

A klasszikus szabályozó elkészítése után a következő lépés egy megerősítéses tanuláson alapuló szabályozó kialakítása volt. Az volt a célom, hogy a korábban elkészített nemlineáris robotmodellt ne változtassam meg, hanem ugyanazon a modellen vizsgáljam meg, hogy egy tanulásalapú szabályozó milyen eredményeket tud elérni.

A megerősítéses tanulási környezetben ezért ugyanazt a három szabadságfokú robotmodellt használtam, amely a klasszikus szabályozó fejlesztésének alapját is képezte. A modell továbbra is tartalmazza a hosszirányú mozgást, a főtest dőlését és a yaw mozgást, valamint a korábban beépített súrlódási és egyéb veszteségi hatásokat.

Az RL szabályozási feladatot ugyanakkor a robot egyensúlyozására és hosszirányú mozgására korlátoztam. Az ágens feladata tehát az volt, hogy a robotot a függőleges, $\theta = 0$ helyzet közelében tartsa, miközben a robot sebessége és pozíciója a megadott referenciaértékeket követi.

A tanulásalapú szabályozó közvetlenül a két kerék közös hajtónyomatékát, vagyis a $\tau_c$ értékét határozza meg:

$$a = \tau_c. \tag{3.74}$$

A beavatkozó nyomatékot ugyanúgy korlátoztam, mint a klasszikus szabályozó esetében:

$$-2,\mathrm{Nm} \leq \tau_c \leq 2,\mathrm{Nm}. \tag{3.75}$$

Ezzel biztosítható, hogy a két szabályozási módszer azonos beavatkozási korlátok mellett legyen később összehasonlítható.

A robot fordulását meghatározó $\tau_d$ különbségi nyomaték nem része a jelenlegi megerősítéses tanulási feladatnak. Az RL vizsgálatok során ezért a yaw referenciaértéke nulla, és a különbségi hajtónyomatékot

$$\tau_d = 0 \tag{3.76}$$

értéken tartottam. A yaw dinamika ettől függetlenül továbbra is része maradt a teljes nemlineáris modellnek. A vizsgálatot azért szűkítettem a hosszirányú mozgásra és az egyensúlyozásra, hogy a klasszikus és a tanulásalapú szabályozó ugyanazon jól meghatározott feladaton legyen összehasonlítható.

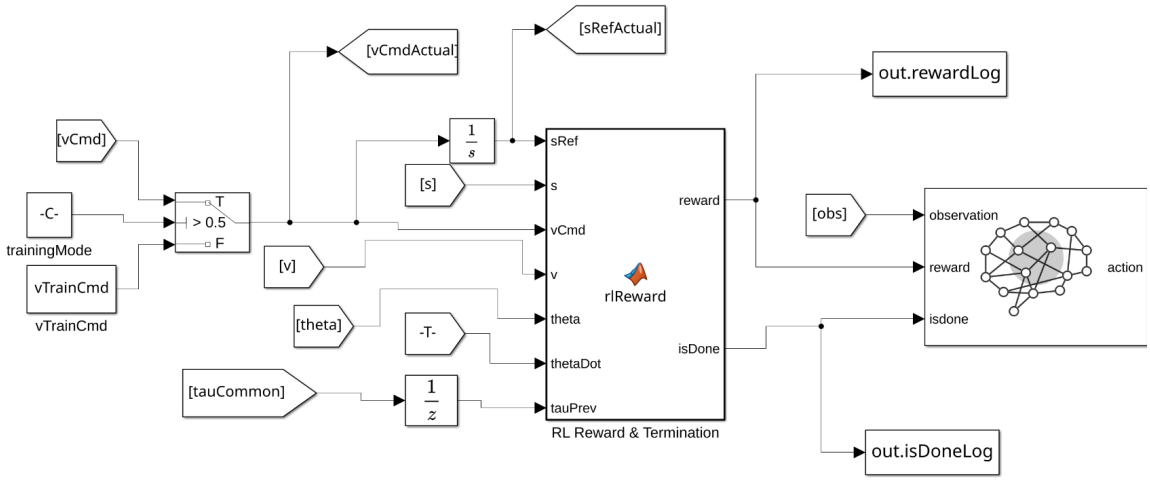
A szabályozó diszkrét időközönként avatkozik be a rendszer működésébe. A megerősítéses tanulási környezet mintavételi idejét

$$T_s = 0{,}02,\mathrm{s} \tag{3.77}$$

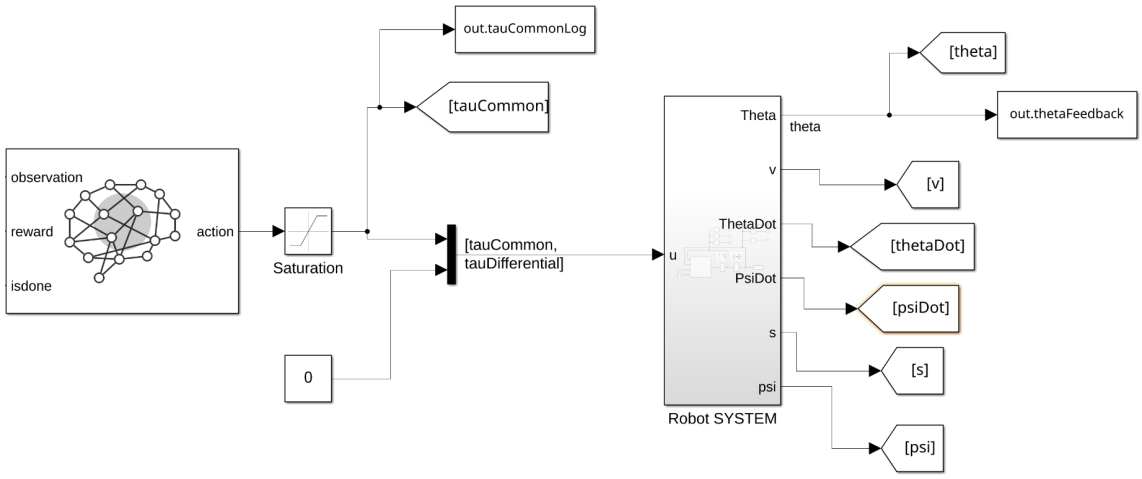
értékben határoztam meg, vagyis az ágens másodpercenként 50 alkalommal kap új megfigyelést a rendszerről és állít elő új beavatkozó jelet.

A kialakított szabályozási körben az ágens minden időlépésben megkapja a robot aktuális állapotából és a referenciajelekből összeállított megfigyelési vektort. A tanítás során alkalmazott referenciajelek, valamint a jutalom- és terminációs feltételek kialakítása a 3.8. ábrán látható.

Az ágens a megfigyelések és a kapott jutalom alapján állítja elő a következő beavatkozó jelet. Az ágens kimenete egy szaturációs blokkon keresztül a robotmodell $\tau_c$ közös hajtónyomaték bemenetére kerül. A különbségi nyomaték értékét az RL vizsgálatok során $\tau_d = 0$ értéken tartottam. Az ágens és a robotmodell közötti kapcsolat a 3.9. ábrán látható.




3.8. ábra. A megerősítéses tanulási környezet referenciajeleinek, jutalomfüggvényének és ágensbemeneteinek kialakítása.




3.9. ábra. Az RL ágens kapcsolata a beavatkozó jellel és a nemlineáris robotmodellel.

### 3.4.2. Megfigyelési tér és beavatkozó jel

A megerősítéses tanulási környezet kialakításánál meg kellett határozni, hogy az ágens milyen információkat kapjon a robot aktuális állapotáról és a kívánt mozgásról. A cél az volt, hogy az ágens rendelkezzen minden olyan információval, amely szükséges az egyensúly megtartásához, valamint a sebesség- és pozícióreferencia követéséhez.

A végleges megfigyelési vektor hat elemből áll:

$$\mathbf{o} = \begin{bmatrix} s_{\text{ref}} & s & v_{\text{cmd}} & v & \theta & \dot{\theta} \end{bmatrix}. \tag{3.78}$$

A megfigyelési vektorban $s_{\text{ref}}$ a kívánt pozíció, $s$ a robot aktuális pozíciója, $v_{\text{cmd}}$ a kívánt előremeneti sebesség, $v$ pedig a tényleges sebesség. Ezek mellett az ágens megkapja a robot $\theta$ dőlésszögét és $\dot{\theta}$ dőlési szögsebességét is.

A dőlésszög és a dőlési szögsebesség elsősorban az egyensúlyozási feladathoz szükséges. A sebesség és a pozíció megadásával az ágens nemcsak a robot függőleges helyzetét tudja stabilizálni, hanem a kívánt hosszirányú mozgást is figyelembe tudja venni.

A sebességparancsból a környezetben integrálással állítom elő a hozzá tartozó pozícióreferenciát:

$$s_{\text{ref}}(t) = \int v_{\text{cmd}}(t), dt. \tag{3.79}$$

Így például állandó pozitív sebességparancs esetén a pozícióreferencia folyamatosan növekszik. Ennek segítségével az ágens egyszerre kap információt a kívánt sebességről és arról is, hogy a robotnak az adott időpillanatban körülbelül hol kellene tartania.

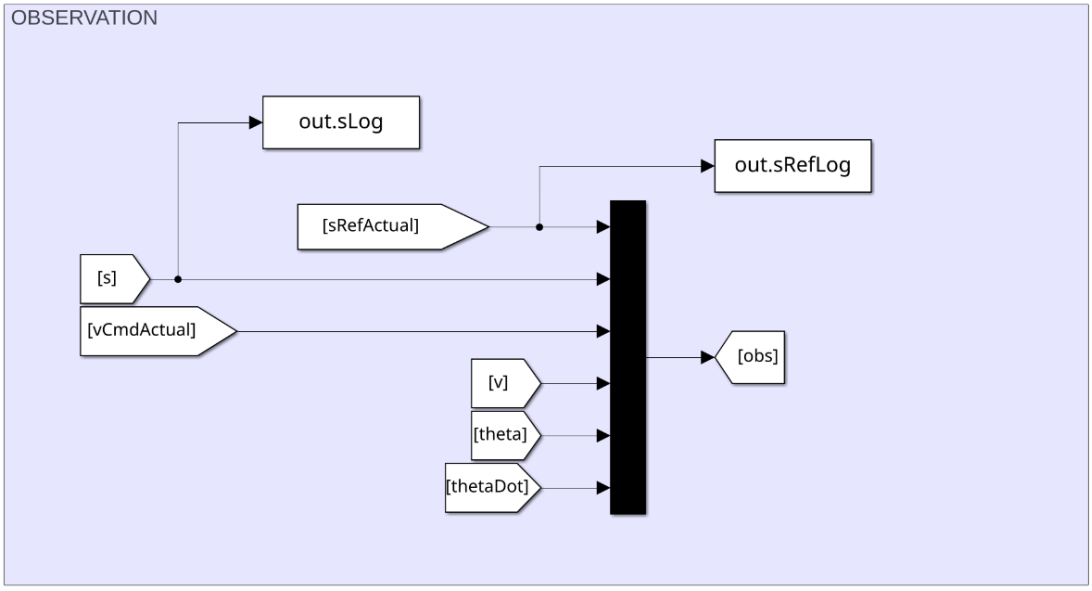
A szabályozó egyetlen kimenete a két kerék közös hajtónyomatéka:

$$a = \tau_c. \tag{3.80}$$

Az akció folytonos értéket vehet fel a

$$-2, \text{Nm} \leq \tau_c \leq 2, \text{Nm} \tag{3.81}$$

tartományban. Az RL Agent által előállított érték egy szaturációs blokkon keresztül kerül a robotmodell bemenetére, így az ágens nem tud a megadott fizikai korlátnál nagyobb hajtónyomatékot létrehozni.




3.10. ábra. Az RL ágens megfigyelési vektorának összeállítása.

### 3.4.3. A jutalomfüggvény kialakítása

A megerősítéses tanulás során az ágens működését a jutalomfüggvény határozza meg. A jutalomfüggvény kialakításánál az volt a célom, hogy az ágens egyszerre tanulja meg a robot függőleges helyzetének megtartását, a kívánt sebesség és pozíció követését, valamint lehetőség szerint ne használjon indokolatlanul nagy hajtónyomatékot.

Mivel a jutalomfüggvényben szereplő mennyiségek különböző nagyságrendűek és különböző mértékegységűek, ezért először normalizált hibaváltozókat alakítottam ki. A pozíció- és sebességhiba:

$$e_s = \frac{s_{\text{ref}} - s}{0{,}5}, \tag{3.82}$$

$$e_v = \frac{v_{\text{cmd}} - v}{0{,}2}. \tag{3.83}$$

A dőlésszög normalizált értékét 10°-hoz viszonyítottam:

$$\theta_N = \frac{\theta}{10^\circ}, \tag{3.84}$$

míg a dőlési szögsebességnél:

$$\dot{\theta}_N = \frac{\dot{\theta}}{2}. \tag{3.85}$$

A beavatkozó nyomaték normalizált értéke:

$$\tau_N = \frac{\tau_{\text{prev}}}{2}. \tag{3.86}$$

A végleges jutalomfüggvényben ezek négyzetes hibái szerepelnek különböző súlyokkal:

$$r = 1 - \left(e_s^2 + e_v^2 + 4\theta_N^2 + 0{,}25\dot{\theta}_N^2 + 0{,}02\tau_N^2\right). \tag{3.87}$$

A legnagyobb súlyt a dőlésszög kapta, mivel az elsődleges feladat továbbra is a robot egyensúlyának megtartása. A pozíció- és sebességhiba azonos súllyal szerepel, így az ágensnek a kívánt mozgást is követnie kell. A dőlési szögsebesség külön büntetése a gyors lengések csökkentését segíti.

A hajtónyomatékhoz tartozó büntetés súlya ehhez képest kicsi. Ennek célja nem az volt, hogy az ágens mindenáron kis nyomatékot használjon, hanem hogy két hasonlóan jó megoldás közül lehetőség szerint azt válassza, amelyhez kisebb beavatkozás szükséges.

A jutalomfüggvényben nem a pillanatnyi $\tau_c$ nyomaték, hanem az előző időlépéshez tartozó $\tau_{\text{prev}}$ érték szerepel. Ehhez a Simulink modellben egy Unit Delay blokkot alkalmaztam. Erre azért volt szükség, mert a pillanatnyi akció közvetlen visszavezetése a jutalomfüggvényre algebrai hurkot hozott volna létre az RL Agent és a reward számítása között. A robotmodell ettől függetlenül mindig az aktuális, késleltetés nélküli $\tau_c$ nyomatékot kapja.

A normál jutalmazás mellett egy terminációs feltételt is meghatároztam. Amennyiben a robot dőlésszögének abszolút értéke meghaladja a

$$|\theta| > 30^\circ \tag{3.88}$$

értéket, az adott tanítási epizód befejeződik. Ebben az esetben az ágens

$$r = -100 \tag{3.89}$$

jutalmat kap. Ez egyértelműen bünteti azokat az eseteket, amikor az ágens elveszíti a robot egyensúlyát.

A jutalomfüggvény végleges kialakítása több tanítási próbálkozás és módosítás eredménye volt. A kezdeti változatoknál elsősorban az egyensúly megtartása dominált, majd a későbbi változatokban nagyobb szerepet kapott a sebesség- és pozíciókövetés is. A végleges változatot a később bemutatott SAC ágens tanításánál használtam.

```
function [reward, isDone] = rlReward( ...
    sRef, s, vCmd, v, theta, thetaDot, tauPrev)

    %% Normalizálási skálák
    sScale        = 0.5;          % [m]
    vScale        = 0.2;          % [m/s]
    thetaScale    = 10*pi/180;    % [rad]
    thetaDotScale = 2.0;          % [rad/s]
    tauScale      = 2.0;          % [Nm]

    %% Normalizált hibák / állapotok
    eS        = (sRef - s) / sScale;
    eV        = (vCmd - v) / vScale;
    thetaN    = theta / thetaScale;
    thetaDotN = thetaDot / thetaDotScale;
    tauN      = tauPrev / tauScale;

    %% Cost
    cost = ...
          1  * eS^2 ...
        + 1.0  * eV^2 ...
        + 4.0  * thetaN^2 ...
        + 0.25 * thetaDotN^2 ...
        + 0.02 * tauN^2;

    %% Reward
    reward = 1.0 - cost;

    %% Elesés
    fallAngle = 30*pi/180;

    isDone = abs(theta) > fallAngle;

    if isDone
        reward = -100;
    end
end
```

3.11. ábra. A jutalom és az epizód befejezési feltételének számítása a Simulink modellben.

## 3.5. Soft Actor-Critic ágens kialakítása és tanítása

A megerősítéses tanulási környezet kialakítása után a következő lépés a Soft Actor-Critic ágens létrehozása és betanítása volt. A tanítás teljes egészében MATLAB/Simulink környezetben, a Reinforcement Learning Toolbox használatával történt. Az ágens ugyanazt a nemlineáris robotmodellt használta környezetként, amelyen korábban a klasszikus szabályozót is kialakítottam.

### 3.5.1. Az SAC ágens felépítése

Az SAC algoritmust azért választottam, mert a szabályozási feladat folytonos beavatkozási terű. A szabályozó kimenete nem néhány előre meghatározott érték közül választ, hanem a megengedett tartományon belül tetszőleges hajtónyomatékot állíthat elő.

Az ágens bemenetét a korábban meghatározott hat elemű megfigyelési vektor adja:

$$\mathbf{o} = \begin{bmatrix} s_{\text{ref}} & s & v_{\text{cmd}} & v & \theta & \dot{\theta} \end{bmatrix}^T. \tag{3.90}$$

Az ágens egyetlen kimenete a két kerék közös hajtónyomatéka:

$$a = \tau_c, \tag{3.91}$$

amelynek megengedett tartománya:

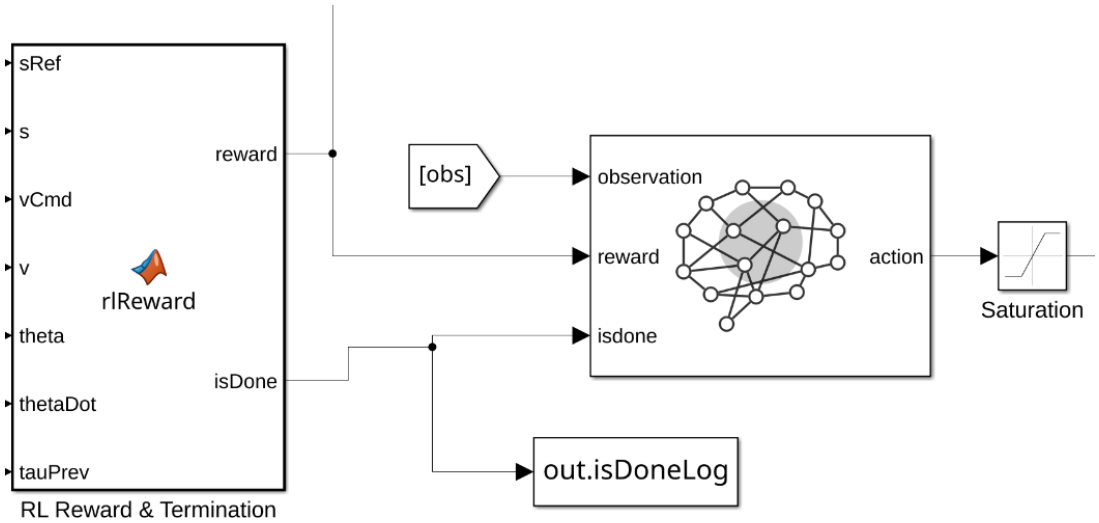
$$-2\,\text{Nm} \leq \tau_c \leq 2\,\text{Nm}. \tag{3.92}$$

A robot fordulását meghatározó $\tau_d$ különbségi nyomaték a tanítás során nulla értéken maradt, vagyis az SAC ágens kizárólag az egyensúlyozásért és a hosszirányú mozgásért felelt.

Az SAC felépítésének megfelelően az ágens actor és critic részekből áll. Az actor feladata a pillanatnyi megfigyelések alapján a beavatkozó jel meghatározása, míg a critic a tanítás során azt értékeli, hogy az adott állapotban kiválasztott akció hosszabb távon mennyire kedvező. A szabályozási lépések mintavételi ideje

$$T_s = 0{,}02\,\text{s} \tag{3.93}$$

volt, vagyis az ágens másodpercenként 50 alkalommal állított elő új beavatkozó jelet.




3.12. ábra. A Soft Actor-Critic ágens kapcsolata a Simulink környezettel.

### 3.5.2. Tanítási környezet és kezdeti feltételek

A tanítás során nem egyetlen rögzített kezdeti állapotból indítottam minden epizódot. Ennek az volt a célja, hogy az ágens ne kizárólag egy meghatározott kezdeti helyzet stabilizálását tanulja meg.

A robot kezdeti dőlésszögét minden tanítási epizód elején véletlenszerűen választottam a

$$-5^\circ \leq \theta_0 \leq 5^\circ \tag{3.94}$$

tartományból. Ez azt jelentette, hogy az ágensnek előre és hátrafelé történő kezdeti dőlésből is vissza kellett állítania a robotot a függőleges helyzet közelébe.

A tanítás során alkalmazott sebességparancs szintén változó volt. A tanítási referencia véletlenszerűen került meghatározásra a

$$-0{,}15\,\mathrm{m/s} \leq v_{\mathrm{cmd}} \leq 0{,}15\,\mathrm{m/s} \tag{3.95}$$

tartományban.

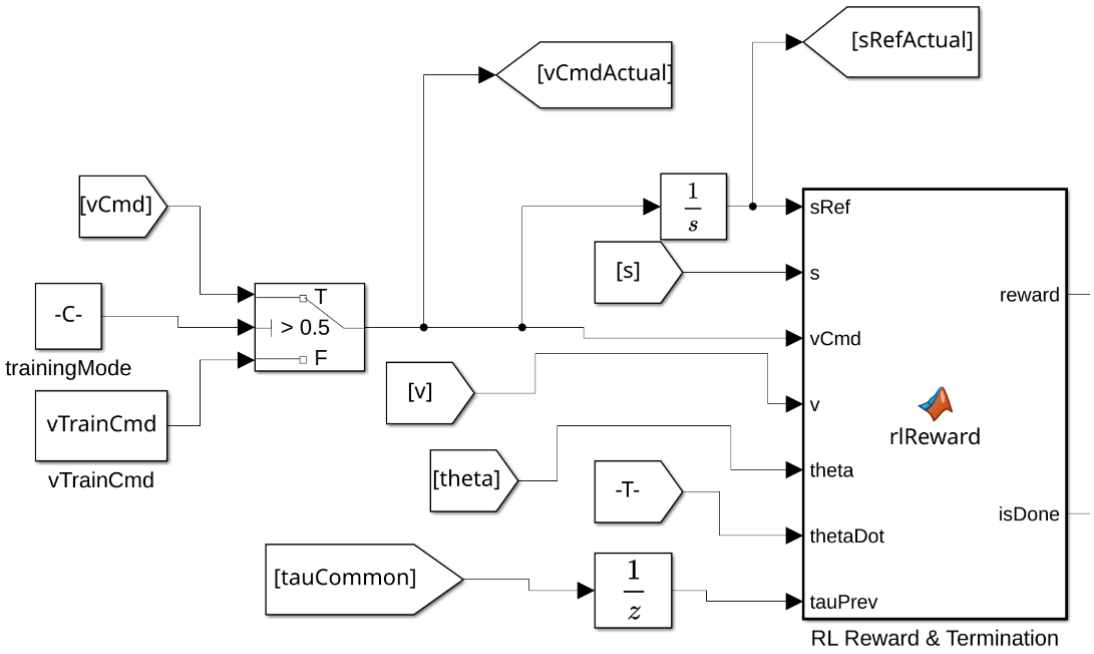
Ez lehetővé tette, hogy az ágens ne csak álló helyzetben történő egyensúlyozást tanuljon, hanem pozitív és negatív irányú mozgás mellett is találkozzon különböző állapotokkal.

A kiválasztott sebességparancsból integrálással állítottam elő a hozzá tartozó pozíció-referenciát:

$$s_{\mathrm{ref}}(t) = \int v_{\mathrm{cmd}}(t)\,dt. \tag{3.96}$$

Így a sebesség- és pozícióreferencia egymással összhangban változott a tanítás során.

A Simulink modellben külön választottam a kézi teszteléshez és a tanításhoz használt referenciajeleket. A `trainingMode` változó határozta meg, hogy a rendszer a kézzel megadott parancsot vagy a tanítás során generált véletlenszerű sebességreferenciát használja.




3.13. ábra. A tanítás során alkalmazott referenciajelek előállítása.

Az epizód azonnal befejeződött, ha a robot dőlésszöge meghaladta a korábban meghatározott

$$|\theta| > 30^\circ \tag{3.97}$$

határértéket. Ezzel elkerülhető volt, hogy az ágens egy már elvesztett egyensúlyi állapotból hosszú ideig folytassa az adott epizódot.

### 3.5.3. A tanítás során végzett módosítások

A végleges tanítási környezet nem az első változatban alakult ki. A fejlesztés során több problémát is tapasztaltam, ezért az observation, a jutalomfüggvény és a referenciajelek kezelését több lépésben módosítottam.

Az első tanítási próbálkozások során elsősorban az egyensúly megtartására és a sebesség követésére helyeztem a hangsúlyt. A pozíció ekkor még nem kapott megfelelő szerepet a tanításban. A robot képes volt az egyensúly megtartására és a sebesség követésére, azonban hosszabb működés során a pozíciója fokozatosan el tudott sodródni.

Ennek javítására a későbbi változatban a robot aktuális pozícióját és a pozíció referenciaértékét is hozzáadtam a megfigyelési térhez. A jutalomfüggvényben szintén megjelent a pozícióhiba büntetése. Ezzel az ágens számára már nemcsak az vált fontossá, hogy megfelelő sebességgel mozogjon, hanem az is, hogy hosszabb távon a kívánt pozícióhoz tartson.

A tanítás további vizsgálata során egy referenciajel-kezelési hibát is találtam. Az egyik korábbi modellváltozatban a jutalomfüggvény a tanításhoz generált véletlenszerű sebességparancsot kapta, miközben az ágens megfigyelési vektorában nem minden esetben ugyanez

a referenciaérték szerepelt. Így előfordulhatott, hogy az ágens egy olyan referencia alapján kapott jutalmat vagy büntetést, amelyet a saját bemenetén nem látott.

A hiba javításakor létrehoztam egy közös, ténylegesen alkalmazott $v_{\mathrm{cmd}}$ referenciajelet. Ezt a jelet ezután ugyanúgy megkapta

- az ágens megfigyelési vektora,
- a jutalomfüggvény,
- valamint a pozícióreferenciát előállító integrátor.

Ezzel a tanítás minden része ugyanazt a referenciajelet használta.

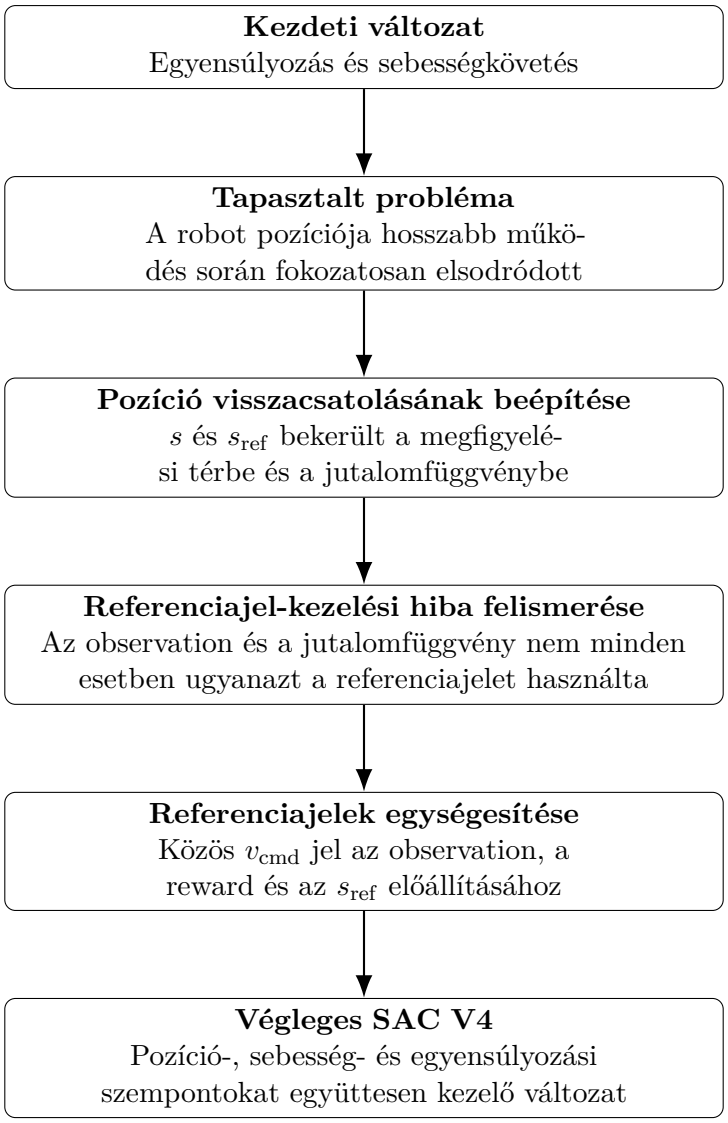
A jutalomfüggvény kialakításánál további problémát jelentett a pillanatnyi hajtónyomaték visszacsatolása. Ha az aktuális $\tau_c$ jel közvetlenül bekerült volna a reward számításába, akkor a Simulink modellben algebrai hurok alakult volna ki az ágens kimenete és a jutalomfüggvény között.

Ennek megoldására kizárólag a jutalom számításánál egy Unit Delay blokkot alkalmaztam, így a jutalomfüggvény az előző szabályozási lépéshez tartozó

$$\tau_{\mathrm{prev}} \tag{3.98}$$

értéket használta. Fontos, hogy ez a késleltetés csak a jutalom számításában szerepelt. A robotmodell minden esetben az ágens aktuális, késleltetés nélküli nyomatékát kapta.

A tanítás során a jutalomfüggvény súlyait is több alkalommal módosítottam. A cél az volt, hogy az egyensúly megtartása továbbra is elsődleges legyen, de közben a sebesség- és pozíciókövetés is megfelelő hangsúlyt kapjon. Ennek eredményeként alakult ki a korábban bemutatott végleges jutalomfüggvény.




3.14. ábra. A megerősítéses tanulási környezet fejlesztésének főbb lépései.

### 3.5.4. A végleges ágens kiválasztása

A tanítási környezet és a jutalomfüggvény módosításai után alakult ki a végleges, V4 jelölésű változat. Ebben már a pozíció- és sebességkövetés, az egyensúlyozás, valamint a beavatkozó nyomaték büntetése is szerepelt, és a referenciajelek kezelése is egységesen történt.

A végleges ágenst összesen 700 tanítási epizódig tanítottam. A tanítás eredményét

`RL_SAC_V4_positionW1_700ep.mat`

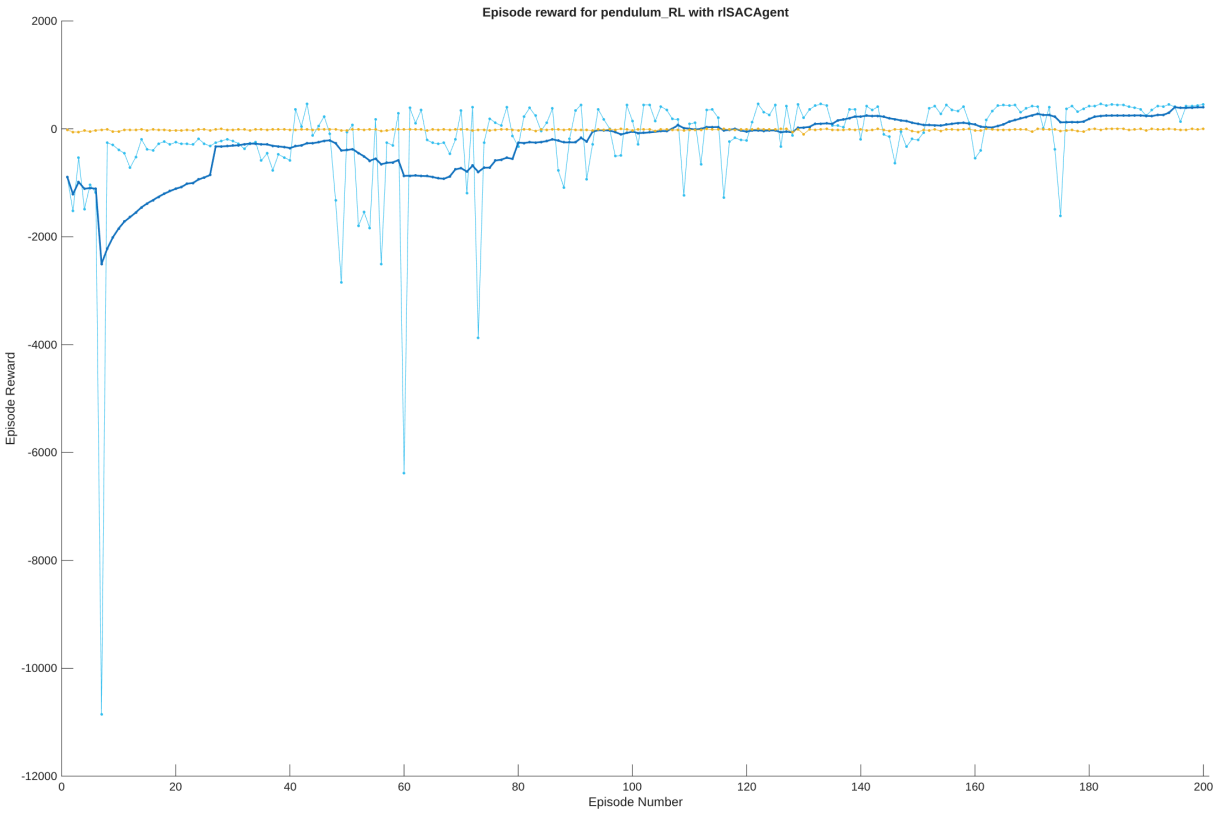
fájlban mentettem el.

Ezt a változatot választottam ki a további vizsgálatokhoz, és ezt használtam később a klasszikus szabályozóval történő összehasonlítás során is.

A tanítás befejezése után a kiválasztott ágenst már nem tanítottam tovább, hanem külön validációs futásokon vizsgáltam. Ennek célja az volt, hogy a végleges szabályozót

olyan rögzített teszteseteken értékeljem, amelyek közvetlenül összehasonlíthatók a klasszikus szabályozó eredményeivel.

A részletes validációs és összehasonlító eredményeket a dolgozat későbbi fejezete tartalmazza.




3.15. ábra. A végleges SAC ágens 700 epizódos tanításának folyamata.

## 3.6. Az összehasonlító vizsgálat kialakítása

### 3.6.1. Azonos vizsgálati feltételek

### 3.6.2. Vizsgálati esetek és mérőszámok

### 3.6.3. Számítási igény vizsgálata

# Irodalomjegyzék


[1] Ronald Ping Man Chan, Karl A. Stol és C. Roger Halkyard. "Review of modelling and control of two-wheeled robots". *Annual Reviews in Control* 37.1 (2013), 89–103. old. DOI: 10.1016/j.arcontrol.2013.03.004.

[2] Sangtae Kim és SangJoo Kwon. "Dynamic Modeling of a Two-wheeled Inverted Pendulum Balancing Mobile Robot". *International Journal of Control, Automation, and Systems* 13.4 (2015). DOI: 10.1007/s12555-014-0564-8.

[3] Huaqiang Zhang és Norzalilah Mohamad Nor. "Control Strategies for Two-Wheeled Self-Balancing Robotic Systems: A Comprehensive Review". *Robotics* 14.8 (2025), 101. old. DOI: 10.3390/robotics14080101.

[4] Wei An és Yangmin Li. "Simulation and control of a two-wheeled self-balancing robot". *2013 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. 2013, 456–461. old. DOI: 10.1109/ROBIO.2013.6739501.

[5] S. Yuan, G. Lei és X. Bin. "Dynamic modeling and sliding mode controller design of a two-wheeled self-balancing robot". *2016 IEEE International Conference on Mechatronics and Automation*. 2016, 2437–2442. old. DOI: 10.1109/ICMA.2016.7558948.

[6] Kiam Heong Ang, Gregory Chong és Yun Li. "PID Control System Analysis, Design, and Technology". *IEEE Transactions on Control Systems Technology* 13.4 (2005), 559–576. old. DOI: 10.1109/TCST.2005.847331.

[7] To Duc Nguyen. "Controlling Two-Wheeled Self-Balancing Robot Moving on Inclined Plane". *VNUHCM Journal of Science and Technology Development* 20.K3 (2017), 45–52. old. DOI: 10.32508/stdj.v20iK3.1091.

[8] Oluwasegun Ayokunle Somefun, Kayode Akingbade és Folasade Dahunsi. "The Dilemma of PID Tuning". *Annual Reviews in Control* 52 (2021), 65–74. old. DOI: 10.1016/j.arcontrol.2021.05.002.

[9] Richard S. Sutton és Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2. kiad. MIT Press, 2018.

[10] Jens Kober, J. Andrew Bagnell és Jan Peters. "Reinforcement Learning in Robotics: A Survey". *The International Journal of Robotics Research* 32.11 (2013), 1238–1274. old. DOI: 10.1177/0278364913495721.

[11] Timothy P. Lillicrap és tsai. "Continuous Control with Deep Reinforcement Learning". *arXiv preprint arXiv:1509.02971* (2015).

[12] Tuomas Haarnoja és tsai. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". *Proceedings of the 35th International Conference on Machine Learning*. 80. köt. Proceedings of Machine Learning Research. PMLR, 2018, 1861–1870. old.

[13] Tuomas Haarnoja és tsai. “Soft Actor-Critic Algorithms and Applications”. *arXiv preprint arXiv:1812.05905* (2018).

[14] Lucian Buşoniu és tsai. “Reinforcement Learning for Control: Performance, Stability, and Deep Approximators”. *Annual Reviews in Control* 46 (2018), 8–28. old. DOI: 10.1016/j.arcontrol.2018.09.005.

[15] Rubén Hernández, Ramon Garcia-Hernandez és Francisco Jurado. “Stabilization of a Two-Wheeled Self-Balancing Robot Using Reinforcement Learning”. *2024 XXVI Robotics Mexican Congress (COMRob)*. 2024. DOI: 10.1109/COMRob64055.2024.10777426.